

BOTTOM-UP PARSING AND PRECEDENCE PARSERS

A bottom-up parser reconstructs a right-most derivation in reverse, conceptually starting with a sentence in the language and ending with the goal symbol. We have previously examined bottom-up parsing as a tree-building process. The key to the process is the identification of the handle in any given right-most sentential form and a production that belongs with the handle in that form.

A bottom-up parser can always be constructed for a context-free grammar. It consists, as for the top-down parser, of an input list, a finite control, and a push-down stack. In general, it is nondeterministic, and there may or may not exist a lookahead system through which a deterministic parser may be constructed. There also exist grammars for which no deterministic parser can be constructed.

There are two broad classes of deterministic bottom-up parsers— the so-called *precedence* parsers and the $LR(k)$ parsers. Of these two, the $LR(k)$ parsers are considerably more powerful; they accept a much larger class of grammars. The $LR(k)$ grammars also include all the $LL(k)$ grammars, and constitute the largest class of deterministic grammars with left-to-right parsing and a k -symbol lookahead. Consequently, given any context-free grammar, it is more likely to have an $LR(k)$ parser than any other.

A bottom-up parser appears to be more complicated than a top-down parser. The parsing theory is somewhat more difficult to follow than for a top-down parser, and the semantic operations seem to be less obvious. We can combat this feeling only by studying a complete compiler system. We shall present not only the bottom-up parsing theory and system, but a generalized semantics system as well, with enough rules so that anyone should be able to design a complete compiler from a set of objectives. Bottom-up semantics may be used with any bottom-up parsing method. The parser need only produce an ordered list of production numbers.

5.1. Nondeterministic Bottom-up Parsing

A bottom-up, nondeterministic parser differs from a top-down, nondeterministic parser in two respects:

1. The stack is more conveniently oriented with its top on its right end. The stack then holds some left suffix of a sentential form at any given

time. The handle will appear on the top of the stack just before an apply action.

2. The push-down automaton is extended by permitting a string of zero or more symbols to be popped from the stack in one operation. Recall that in a top-down PDA, it was never necessary to pop more than one symbol.
3. A NDPA constructed from a grammar will have two states. Recall that a top-down automaton contained only one state. However, the added state is used only to provide an elegant halt condition.

An *extended PDA* is a 7-tuple $P = (Q, \Sigma, H, \delta, q_0, \perp, F)$, where:

- Q is a set of *states*.
- Σ is a finite *input alphabet* (the tokens of the language).
- H is a finite *stack alphabet*.
- δ is a *finite mapping* from a 3-tuple in $Q \times H^* \times (\Sigma \cup \{\epsilon\})$ to the finite subsets of $Q \times H^*$.
- q_0 is a *start state* in Q .
- $\perp$ is an *initial stack symbol*, in H .
- F is a set of *halt states*.

Except for δ , this PDA is exactly like the top-down automaton introduced earlier. In this PDA, the mapping function δ considers the present state (in Q), but it may consider the symbol under the read head, and it may consider a string of 0 or more tokens on the top of the stack. In any case, δ is finite; it must be representable as a finite table. Through δ , the state, a top-of-stack string, and the next input token are mapped (in one move) to a new state and the stack top string is replaced by another string.

A *configuration* is written as follows. The stack top is at the right end of the stack:

(state, stack, input list)

Then a *move* is defined as a transformation of one configuration into another, as follows:

$(p, zx, bw) \vdash (q, zy, w)$

where $b \in \Sigma \cup \{\epsilon\}$, x, y , and z are in H^* , $w \in \Sigma^*$, and where

$\delta(p, x, b)$ contains (q, y)

The PDA is *nondeterministic* if some $\delta(p, x, b)$ contains more than one pair, or in other ways, e.g., through an empty x or b .

As before, the *language* defined by P , denoted $L(P)$, is the set

$$\{w \mid (q_0, Z_0, w) \vdash^* (q, x, \epsilon) \text{ for some } q \text{ in } F \text{ and } x \text{ in } H^* \}$$

Note that the stack need not be empty to halt, and that the PDA is defined in such a way that it may make additional moves after its stack is empty.

Equivalence of Extended PDA's and Context-Free Languages

As before, we may define an extended PDA that recognizes the language of any context-free grammar. However, this one can reasonably be said to operate by handle recognition and replacement, making it a bottom-up parser.

Let (N, Σ, P, S) be a context-free grammar G . Then we construct an extended PDA R such that $L(R) = L(G)$ as follows:

1. R will have two states, $\{q, r\}$.
2. R 's input alphabet is Σ .
3. The stack alphabet H consists of $N \cup \Sigma \cup \{\perp\}$.
4. The initial stack symbol is $\perp$.
5. The initial state is q , and the halt set $F = \{r\}$.
6. The mapping δ is defined as follows:
 - (a) *Shift rule*: For every terminal symbol b in Σ , $\{(q, b)\}$ is a member of $\delta(q, \epsilon, b)$. These moves have the effect of shifting terminal symbols from the input source into the stack top. For example, the configuration (q, x, bw) would go to the configuration (q, xb, w) in such a move.
 - (b) *Apply or reduce rule*: For every production $A \rightarrow w$ in P , $\delta(q, w, \epsilon)$ contains (q, A) . These moves have the effect of taking a handle w on the stack top and reducing it to a nonterminal symbol A . Thus, a configuration (q, xw, u) would go to (q, xA, u) in such a move.
 - (c) *Halt rule*: $\delta(q, \perp S, \epsilon)$ contains (r, ϵ) , where S is the start symbol. This move can only occur once for a given parse and results in a halt. Note that it requires a stack containing only " $\perp S$ ", since there is no way for any stack symbols to appear beneath the $\perp$.

Such a parser is sometimes called a *shift-reduce* parser, since most of its moves are based on (6a) or (6b).

An Example. Consider the arithmetic grammar G_0 , given below.

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \\ T &\rightarrow T * F \\ T &\rightarrow F \\ F &\rightarrow (E) \\ F &\rightarrow a \end{aligned}$$

The PDA mapping for this grammar is then as follows:

$$\begin{aligned} \delta(q, \epsilon, a) &= \{(q, a)\} && \text{from rule 6(a)} \\ \delta(q, \epsilon, +) &= \{(q, +)\} \\ \delta(q, \epsilon, *) &= \{(q, *)\} \\ \delta(q, \epsilon, "(") &= \{(q, "(")\} \\ \delta(q, \epsilon, ")") &= \{(q, ")")\} \\ \delta(q, E + T, \epsilon) &= \{(q, E)\} && \text{from rule 6(b)} \\ \delta(q, T, \epsilon) &= \{(q, E)\} \\ \delta(q, T * F, \epsilon) &= \{(q, T)\} \\ \delta(q, F, \epsilon) &= \{(q, T)\} \\ \delta(q, (E), \epsilon) &= \{(q, F)\} \\ \delta(q, a, \epsilon) &= \{(q, F)\} \\ \delta(q, \perp E, \epsilon) &= \{(r, \epsilon)\} && \text{from rule 6(c)} \end{aligned}$$

Although none of the mapping functions contains more than one pair, it is clear that the PDA is nondeterministic. The first set of five rules may be applied at any time, to shift another input symbol into the stack. Similarly, there may be opportunities to apply more than one of the second group of transitions. For example, with $T * F$ on the stack top, the string $T * F$ may be reduced to T , or F may be reduced to T .

Despite the great number of possible sequences of moves, the PDA recognizes exactly the language $L(G)$. Most of the moves for a given string end in blind alleys.

Consider the string " $a * a + a$ ". Only the following sequence of moves can reach the halt state:

1. $(q, \perp, a * a + a) \vdash$
2. $(q, \perp a, * a + a) \vdash$
3. $(q, \perp F, * a + a) \vdash$
4. $(q, \perp T, * a + a) \vdash$
5. $(q, \perp T *, a + a) \vdash$
6. $(q, \perp T * a, + a) \vdash$
7. $(q, \perp T * F, + a) \vdash$
8. $(q, \perp T, + a) \vdash$

9. $(q, \perp E, +a) \vdash$
10. $(q, \perp E+, a) \vdash$
11. $(q, \perp E+a, \epsilon) \vdash$
12. $(q, \perp E+F, \epsilon) \vdash$
13. $(q, \perp E+T, \epsilon) \vdash$
14. $(q, \perp E, \epsilon) \vdash$
15. (r, ϵ, ϵ)

Now consider the seventh configuration, $(q, \perp T^*F, +a)$. It is possible to apply a shift rule to this, yielding $(q, \perp T^*F+, a)$. However, no rule provides for reducing a stack with “+” on top, so we must again shift, yielding $(q, \perp T^*F+a, \epsilon)$. At this point, we may reduce “a” to F, yielding $(q, \perp T^*F^*F, \epsilon)$, and then reduce F to T and T to E, yielding $(q, \perp T^*F^*E, \epsilon)$. However, we are at the end of the line for this particular sequence of moves. No further reductions or shifts are possible, and we have failed to reach the configuration $(q, \perp E, \epsilon)$ that permits a halt. There have also been no other possible moves. We choose this example to illustrate the central point—this PDA recognizes exactly the language defined by the grammar G. A proof follows.

Proof: We prove a somewhat strong result, which can easily be seen to imply that the PDA language and $L(G)$ are equivalent:

Lemma 5.1. $S \Rightarrow^* xAy \Rightarrow^+ zy$ if and only if $(q, \perp, zy) \vdash^* (q, \perp xA, y)$, where the derivation is right-most. A is a nonterminal.

The “only if” part may be proven by induction on the number of steps n in the second sequence of derivations. The lemma states that, given a right-most sentential form xAy , in which A is the right-most nonterminal, and such that xA derives a terminal string z , then the PDA must be able to accept string z through shifts and reduces, and reach a stack that contains xA . The string y is unimportant, except within the inductive assertion of the proof.

For $n = 1$, string x must be empty, and a production $A \rightarrow z$ exists, where z is a terminal string. There clearly exist a sequence of shift moves for the terminal string z , yielding the moves:

$$(q, \perp, zy) \vdash^* (q, \perp z, y)$$

Note that z may also be empty. Then the stack top z may be reduced to A by the PDA construction rules, yielding the result

$$(q, \perp z, y) \vdash (q, \perp A, y)$$

which was to be proven, for the basis.

Now suppose the lemma true for any number of derivation steps less than n and that $xAy \Rightarrow^+ zy$ in n steps. The second sequence of derivations begins with

$$xAy \Rightarrow xwy \Rightarrow^* zy$$

where $A \rightarrow w$ is a production. Clearly, $xw \Rightarrow^* z$ in less than n derivation steps. Now xw may consist solely of terminals, which reduces this to the case $n = 1$. Hence suppose xw contains at least one nonterminal, say B ; we also let B be the right-most nonterminal in xw , without loss of generality; then

$$xw = rBs$$

where s is a terminal string. We now have

$$S \Rightarrow^* xAy \Rightarrow xwy = rBsy \Rightarrow^* zy = usy,$$

since the terminal z must contain s as a suffix. But by the inductive hypothesis,

$$(q, \perp, zy) = (q, \perp, usy) \vdash^* (q, \perp rB, sy)$$

Then by some shifting rules,

$$(q, \perp rB, sy) \vdash^* (q, \perp rBs, y)$$

Next, since $rBs = xw$,

$$(q, \perp xw, y) \vdash (q, \perp xA, y)$$

QED.

A proof of the “if” part is left for an exercise.

Exercises

1. Construct a bottom-up NDPA for the following grammar G_0' :

$$E \rightarrow E + E \mid E * E \mid a \mid (E)$$

This grammar is ambiguous. Demonstrate that its NDPA will accept the string $a + a * a$ by either of the two derivations for it in G_0' .

2. Complete the proof of lemma 5.1.

Deterministic Bottom-up Parsing

The PDA constructed from a grammar G described in the last section is nondeterministic for several reasons:

1. A shift rule can always be applied as long as there exist remaining symbols in the input list.
2. Although a reduction rule can only be applied when the top of a stack string matches a right part of some production, there may be several

different productions whose right parts match a stack top string. Thus, for a stack top string xy , any production $A \rightarrow y$ might be applied through a reduction rule to yield xA . Note that the length of y is not defined by the PDA.

3. As a special case of point 2, an empty production $A \rightarrow \epsilon$ could conceivably be applied at any time, since it requires no stack top match at all. The mapping associated with such a production is $\delta(q, \epsilon, \epsilon)$ contains (q, A) , i.e., nonterminal A is pushed onto the stack.

There are two commonly used bottom-up, deterministic PDA—the precedence parsers and the LR(k) parsers. Each of them has a number of variations that may be employed to increase its power or to reduce its size.

5.2. Precedence Parsing

A precedence parser identifies the handle within a right-most sentential form without identifying the production. A *simple precedence parser* associates one of the three relations, “<”, “>”, or “=” with each pair of adjacent tokens in a sentential form. For a suitable grammar, the handle always becomes delimited between “<” and “>”, and within it, “=” applies to the pairs. Finally, in the string preceding the handle, “<” or “=” applies, but not “>”. The handle may therefore be identified by scanning a sentential form from left to right until “>” is seen, then from right to left until “<” is seen.

A typical precedence table is given in figure 5.1, for a simple grammar G_2 :

$$\begin{array}{l} E \rightarrow E - T \mid T \\ T \rightarrow (E) \mid a \end{array}$$

Every sentence is preceded and followed by a special symbol “ $\perp$ ”, in order that the first and last pairs have a meaningful precedence relation. The table of figure 5.1 contains one entry for which two relations hold: $\{ (, E \}$ is in both of the relations “<” and “=”. This is called a *conflict* and means that the parser is nondeterministic.

For example, in grammar G_2 , within the sentential form $\perp((E-T))\perp$, the adjacent pairs carry the following precedence symbols:

$$\perp < (< (\leq E = - = T >) \dots$$

since $E - T$ is the handle of this sentential form. The “ $\leq$ ” conflict for $\{ (, E \}$ indicates that “ $(E-T)$ ” could be a handle; however, this fits no production right part and therefore cannot be a handle.

The pairs in the string past the handle are not important, since the object is

		Right						
		-	(	)	a	E	T	⊥
Left	-		<		<		=	
	(		<		<	≤	<	
	)	>		>				>
	a	>		>				>
	E	=		=				>
	T	>		>				>
	⊥		<		<	<	<	

$$G_2: E \rightarrow E - T \mid T$$

$$T \rightarrow (E) \mid a$$
Figure 5.1. Simple precedence relation table for grammar G_2 , shown.

to delimit the handle. Once the handle is reduced by applying a production (in this case, $E \rightarrow E + T$), a new sentential form is obtained, and its handle may similarly be found.

Although this appears to be an impossibly weak system, it is sufficiently powerful to parse (with a few special problems) most common programming languages, such as Algol, Fortran, and Basic. We shall therefore develop simple precedence in more detail.

A parsing table can be systematically constructed from a basis grammar. If the parsing table is free of conflicts, and no two productions have the same right member, then the grammar is a *simple precedence* grammar, and the precedence parser will work correctly. If conflicts exist, it may be possible to extend the precedence relations to include more than one symbol, or it may be possible to resolve the conflicts by modifying the parsing method slightly. Otherwise, the grammar is either ambiguous or unsuitable as a basis for a precedence parser.

Exercises

1. Find the handle in each of the following sentential forms, using the precedence table, figure 5.1:

$$\begin{aligned} &\perp(E)a\perp \\ &\perp((a-a))\perp \\ &\perp T\perp \\ &\perp(E-T)\perp \end{aligned}$$

2. Show that each of the strings in exercise 1 are sentential forms by (a) displaying a derivation, and (b) by reducing the handle, finding the next handle, etc., until $\perp E \perp$ is obtained. Resolve the “(, E” conflict in favor of the longest handle.
3. What happens upon parsing each of the following strings using figure 5.1? Are these sentential forms in G_2 ?

$$\begin{aligned} &\perp(-a)\perp \\ &\perp E)\perp \end{aligned}$$

5.2.1. Relations

In order to discuss the precedence parsers in more detail, we need some background development.

A *relation* on two sets P and Q is some set of ordered pairs (x, y) such that x is in P and y is in Q . We may write

$$x R y$$

to indicate that the ordered pair (x, y) is in relation R . We may also write: $y \in R(x)$. An example of a relation is equality on pairs of integers; $I = J$ is true if the integer I is the same as J and false otherwise. The members of the integer equality relation are $(1, 1), (2, 2), \dots$. Another example of a relation is the set of productions in a grammar G ; P is a subset of ordered pairs drawn from the sets N (nonterminals) and $(N \cup \Sigma)^*$. This relation is indicated “ $\rightarrow$ ”.

A relation R is said to be *transitive* if (A, B) in R and (B, C) in R imply that (A, C) is in R , which may also be written:

$$\text{if } (A R B \text{ and } B R C) \text{ then } A R C$$

A relation R is said to be *reflexive* if, for every A in R , (A, A) is in R . A relation R is said to be *symmetric* if (A, B) in R implies that (B, A) is in R .

A relation may be expressed as a matrix consisting of Boolean 0's and 1's. A “1” indicates that the corresponding row and column variables constitute a pair that is in the relation, and a “0” indicates a pair that is not in the relation.

The Boolean matrix for a symmetric relation is diagonally symmetric, i.e., an element at (i, j) in the matrix is equal to the element at (j, i) . The matrix for a reflexive relation contains all 1's along its diagonal.

The *transitive completion* of a relation R , denoted R^+ , is defined as:

1. R is in R^+ .
2. If the pairs (A, B) and (B, C) are in R^+ , then (A, C) is in R^+ .

It can be seen that the inclusion of one pair through rule (2) may precipitate the inclusion of other pairs. The inclusion process must eventually terminate if R is a finite set.

The *reflexive transitive completion* of a relation R , denoted R^* is the union of R^+ and every pair (x, x) such that " x " is in R .

5.2.2. Boolean Matrix Sum and Product

We pointed out in the previous section that a relation can be expressed as a Boolean matrix. Sum and product operations on two matrices are convenient in constructing precedence tables.

The Boolean sum $P = R \vee S$ of two compatible matrices R and S is defined by

$$p(i, j) = r(i, j) \vee s(i, j) \quad \text{for all } (i, j).$$

The Boolean product $P = R \wedge S$ of two square Boolean matrices R and S is defined as follows, where $r(i, j)$ is a member of R , $s(j, k)$ is a member of S , $p(i, k)$ is in P , and the rank of the matrices is n :

$$p(i, k) = \bigvee_{j=1}^n (r(i, j) \wedge s(j, k))$$

The Boolean product is analogous to the algebraic product of matrices, except that a Boolean matrix contains only 0's and 1's. The product matrix element (i, k) is found by multiplying row i of matrix r by column k of matrix s ; a bit-by-bit Boolean product is formed, and the resulting bit is the logical OR of these products.

An example Boolean matrix product is given in figure 5.2.

$$\begin{array}{ccc} \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} & \times & \begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix} \\ \text{(a)} & & \text{(b)} \qquad \qquad \text{(c)} \end{array}$$

Figure 5.2. Boolean matrix product illustrated

Matrix Product and Transitive Completion of a Relation

Now the interesting result: the transitive completion of a relation R may be found by repeatedly replacing R by $R \vee (R \wedge R)$. We will develop a convenient and fast method of constructing precedence tables for a grammar on a digital computer.

This result is a consequence of the following theorem:

Theorem 5.1. Let X and Y be two square Boolean matrices representing the relations R and S , respectively. Then the pair (A, B) is in the product relation $X \wedge Y$ if and only if there exists an element C such that (A, C) is in R and (C, B) is in S .

Proof: The “if” part carries the postulate that C exists. Then matrix X has a “1” at the intersection of row “A” and column “C”, and matrix Y has a “1” at the intersection of row “C” and column “B”. Then the product matrix carries a “1” in row A, column B, and (A, B) is therefore in the product relation.

For the “only if” part, suppose the pair (A, B) is in the product matrix $X \wedge Y$. From the matrix product formula, there must be some row k in Y and column k in X such that there is a “1” at column i in Y and row j in X , otherwise the union could not be “1” for $i = A$ and $j = B$. But this implies that (A, C) is in R and (C, B) is in S , for some C . QED

Warshall's Algorithm

Warshall [1962] found a fast means of computing the transitive completion B^+ of a relation B . His algorithm is expressed by the following Pascal program, where B is some given square Boolean matrix. Matrix B is transformed into its transitive completion.

```

var I, J, K, N: integer;  {N is the rank of the matrix B}
type BOOLMAT = array [1..N, 1..N] of Boolean;
var B: BOOLMAT;

for I:=1 to N do
  for J:=1 to N do
    if B[J, I]=TRUE then
      for K:=1 to N do
        B[J, K]:=B[J, K] ∨ B[I, K];

```

Exercises

1. Consider the set of all living people in a monogamous society that forbids divorce. Classify each of the following relations on the pairs $\{A, B\}$ as reflexive, symmetric, and/or transitive:

- (a) A is an ancestor of B
 - (b) A is the mother of B
 - (c) A is a cousin of B
 - (d) (A is a father of B) or (A is a son of B)
 - (e) A lives in the same house as B
 - (f) A is married if and only if B is married
 - (g) A is single if B is single
 - (h) A is married if and only if B is single
2. Given a relation R expressed as a Boolean matrix, e.g.,

var B: array [1..N, 1..N] of boolean

Write Pascal procedures that:

- (a) Determine if R is reflexive.
 - (b) Determine if R is symmetric.
 - (c) Determine if R is transitive.
3. Consider a finite set of cities C with some interconnecting roads, and let relation R be defined by: “ $A R B$ if and only if there exists a road from A to B that passes through no other city.” (Some of the roads are one-way). What is the significance of:
- (a) R is reflexive.
 - (b) R is symmetric.
 - (c) R is transitive.
 - (d) The transitive completion of R .

Also explain how each of the following sets can be generated:

- (e) *Trap* cities, that can be entered but not left,
- (f) *Exit* cities, that can be left but not entered,
- (g) The largest subsets C' of C such that every city in C' can be reached from every other city in C' .

5.2.3. Viable Prefix

A *viable prefix* is some prefix of a right-most sentential form that may include the handle, but no symbols past the handle. Thus if xhy is a right-most sentential form, such that h is the handle, then a viable prefix of xhy is any string r such that $xh = rs$, where s may be empty.

5.2.4. Precedence Pairs

An (m, n) *precedence pair*, where m and n are integers ≥ 0 , is a pair of strings (x, y) such that $|x| = m$, $|y| = n$, xy is a substring of a right-most sentential form, and x lies wholly within a viable prefix. Note that string y may be within a viable prefix, or may lie partially or wholly within the following string. Since we demand that x have exactly the length m , and y the length n , we prefix every sentential form with m special symbols $\perp$, and suffix every sentential form with n special symbols.

For example, consider the sentential form $((E - T))$ in grammar G_2 , figure 5.1, and let $(m, n) = (2, 1)$. Then we rewrite the form as $\perp \perp ((E - T)) \perp$ so that it is always possible to define a $(2, 1)$ precedence pair. Within this form, since $E - T$ is the handle, the viable prefixes and $(2, 1)$ precedence pairs are as follows.

Viable prefixes:

ϵ
 $($
 $(($
 $((E$
 $((E -$
 $((E - T$

$(2, 1)$ precedence pairs:

$\{\perp \perp, (), \{\perp (, (), \{((, E), \{(E, -), \{E -, T), \{-T,)\}$

5.2.5. Precedence Relations

An (m, n) *precedence relation* is a relation on the set of (m, n) precedence pairs defined on the right-most sentential forms within a grammar G . The three relations ($<$, $>$, $=$) are defined as follows:

1. A pair (x, y) is in the “ $>$ ” relation if and only if $|x| = m$, $|y| = n$, and there exists some sentential form rhs such that $ux = rh$ and $yv = s$, and h is the handle, (r and s are assumed to include the necessary prefixes $\perp \perp \perp \dots$ and suffixes $\perp \perp \perp \dots$). That is, (x, y) marks the boundary position between the end of the handle h and the beginning of the following string s . However, x may include h , or h may include x . Note that y is a terminal string. The three dots “ $\dots$ ” stand for “any string including empty”.

2. A pair (x, y) is in the “ $=$ ” relation if and only if $|x| = m$, $|y| = n$, and there exists some sentential form rhs such that $ux = rh'$ and $yv = h''s$, where the handle h is $h'h''$, and neither h' nor h'' is empty. That is, (x, y) marks a position within a handle, so that at least one tail symbol of x is in the handle and at least one head symbol of y is in the handle. Note that if the handle length is 1 or less, no “ $=$ ” pair is defined for the handle.

3. A pair (x, y) is in the “ $<$ ” relation if and only if $|x| = m$, $|y| = n$, and there exists some sentential form rhs such that h is the handle, $ux = r$ and $yv = hs$. That is, the pair (x, y) marks the beginning of the handle, such that the head of y is the head of the handle, and the tail of x is the tail of the string preceding the handle.

4. For every symbol X such that $S \Rightarrow *X\dots$, the pair $(\perp \dots \perp, X)$ is in $<$. For every symbol X such that $S \Rightarrow \dots *X$, the pair $(X, \perp \dots \perp)$ is in $>$. These special relation members are needed to cover the leading and trailing $\perp$ special symbols added to sentences. S is the start symbol.

5.2.6. Simple Precedence Grammar

If the grammar is such that no two productions have the same right member, then the grammar is said to be *uniquely invertible*. As we have seen, the precedence relations only identify the handle of a sentential form; they do not specify the production associated with that handle. Hence a precedence parser can be constructed for a grammar only if the grammar is uniquely invertible.

A grammar is said to be (m, n) *simple precedence* if and only if the relations “ $<$ ”, “ $>$ ” and “ $=$ ” are pairwise disjoint, and the grammar is uniquely invertible.

Example. Consider the simple grammar G_2 :

$$\begin{aligned} E &\rightarrow E - T \mid T \\ T &\rightarrow (E) \mid a \end{aligned}$$

Note that G_2 is uniquely invertible. The (1, 1) precedence relations “<”, “>”, and “=” are given in figure 5.1. We shall discuss systematic methods of generating such a table later. Let us first consider a short derivation to demonstrate that the precedence relations do in fact enable us to deterministically parse a sentence bottom-up. Consider the string $(a - a) - a$, which is in the language $L(G_2)$. The first group of relations, based on the table, are

$$\perp < (< a > - \dots$$

(The table indicates that the pairs $\{\perp, (\}$ and $\{(\}, a\}$ are in the relation <, and that $\{a, -\}$ is in >). The handle is clearly “a”, which can only be reduced to T. The reduction yields a new sentential form

$$\perp < (< T > - \dots$$

which indicates that T is to be reduced to E, yielding

$$\perp < (< E = - < a >) \dots$$

(The table contains a (<, =) conflict for the pair $\{(\}, E\}$. We shall deal with this matter next. For now, we select the correct choice by a nondeterministic oracle.) Note that although an “=” has appeared, the handle is “a”, delimited by “< ... >”, which indicates a reduction of “a” to T, yielding

$$\perp < (< E = - = T >) \dots$$

The handle here is $E - T$, which reduces to E. The next few steps are:

$$\perp < (= E =) > - \dots \perp$$

$$\perp < T > - \dots \perp$$

$$\perp < E = - < a > \perp$$

$$\perp < E = - = T > \perp$$

$$\perp < E > \perp$$

This terminal sentential form consists of the goal symbol bracketed by the special delimiter symbols $\perp \dots \perp$.

Now let us examine the precedence table itself in the light of these definitions of a (1, 1) precedence relation. Each of the three relations may be determined (in theory at least) by examining all the possible right-most sentential forms. Unfortunately, there are usually an infinite number of them, so that this is hardly practical. Nevertheless, here are some derivations that bring out some of the table entries:

- $\perp E \perp \Rightarrow \perp T \perp$, therefore $\{\perp, T\}$ is in < and $\{T, \perp\}$ is in >.

- $\perp E \perp \Rightarrow \perp E - T \perp$, therefore $\{\perp, E\}$ is in $<$, $\{E, -\}$ is in $=$, and $\{-, T\}$ is in $=$.
- $\perp T \perp \Rightarrow \perp (E) \perp$, therefore $\{\perp, ()\}$ is in $<$, $\{(, E\}$ and $\{E,)\}$ are in $=$, and $\{(, \perp\}$ is in $>$.
- $\perp T \perp \Rightarrow \perp a \perp$, therefore $\{\perp, a\}$ is in $<$, and $\{a, \perp\}$ is in $>$.
- $\perp E - T \perp \Rightarrow \perp E - a \perp$, therefore $\{-, a\}$ is in $<$.

Exercises

1. Find derivations in G_2 that bring out the remaining precedence pairs of figure 5.1.
2. Consider grammar G_2 . Show that the pairs $((, -)$ and (a, E) cannot be in any of the three relations. Show that $(-, a)$ cannot be in $"="$ or $">"$. Hint: study the production tree for G_2 .
3. Design efficient Pascal data structures and an algorithm for a precedence parser. The parser should return apply production numbers, given an input string, and halt on a syntax error.

Blank Entries

What about the blank entries in a precedence table? They apparently mean that the associated pair cannot appear and that therefore any symbol pair showing up in a parse that maps to a blank entry must be because of a syntax error. But how can we prove that a given entry is truly blank? We can be quite ingenious in finding sentential forms to fill in various table values, but we can never be quite sure that we have found them all. We therefore need a more systematic approach to precedence table generation than simply looking at various sentential forms. We take up this matter in section 5.2.7.

Three Theorems on Precedence Relations

Theorem 5.2. No grammar containing an empty production can be a precedence grammar.

Proof: Suppose a grammar G contains an empty rule, $A \rightarrow \epsilon$. Then some derivation

$$S \Rightarrow * xuAvy \Rightarrow xuvy \Rightarrow * \dots$$

exists, where u and v are terminal or nonterminal symbols (not strings). Now the handle of the form $xuvy$ is an empty string between u and v . By the

precedence relation definitions therefore, $u < v$ and $u > v$, which creates a precedence conflict. QED

Theorem 5.3. Given any right-most sentential form xhy , where h is the handle, in a simple precedence grammar G , then either $<$ or $=$ holds between every pair within x .

Proof: Let $xhy = ruvs$, where u and v are some (m, n) precedence pair, and ru is contained in x . Thus (u, v) marks a boundary between symbols in x , or between x and the handle.

Consider the partial derivation tree T for the sentential form $ruvs$, and the nodes N_u and N_v . N_u is associated with the right-most symbol in u , and N_v with the left-most symbol in v . Then consider the upward paths in T from N_u and N_v ; these must intersect in some node N_B associated with nonterminal B .

Suppose first that N_B is not the parent of N_u . There is then a subtree rooted in a child of N_B with a height of at least 1. It therefore contains the handle, contradicting the premise that the handle of $ruvs$ is in vs . Therefore N_B is the parent of N_u .

If N_B is also the parent of N_v , we see that the pair (u, v) is in " $=$ ". If N_B is not a parent of N_v , it is nevertheless an ancestor. We then have a right-most derivation of the form

$$S \Rightarrow^* \dots B \dots \Rightarrow \dots uV \dots \Rightarrow^* \dots uV' \dots \Rightarrow \dots uv \dots$$

and by the precedence definitions, we see that (u, v) is in $<$.

Theorem 5.3 implies that a sentential form may be scanned from left to right through the string that precedes the handle. If the grammar is simple precedence, then only " $<$ " or " $=$ " applies between the pairs of this prefix string, and the scan may continue until the handle is found; it is delimited by " $>$ " on its right end. Note that the fundamental precedence rule 3 does not specify precedence relations between the pairs in the string preceding the handle, and it might have turned out that some of these could be in " $>$ " or in no precedence relation. Either case would be fatal to the prefix scanning operation. We now have assurance that the prefix scanning will work properly.

Theorem 5.4 Let $\perp^m w \perp^n$ be accepted by an (m, n) precedence parser. Then w is a right-most sentential form in G .

Proof: This theorem assures us that an (m, n) precedence parser accepts only strings in $L(G)$. It should be obvious from the previous discussion that a string in $L(G)$ is accepted by a (m, n) precedence parser, since the (m, n)

precedence relations are set up in such a way as to always recognize the handle.

We prove this by induction on the number of steps in the reduction process. For 0 steps, we must have $\perp^m S \perp^n$, where S is the start symbol, obviously a sentential form. Therefore assume the theorem true for all steps less than some k .

Let $\perp^m w \perp^n$ be accepted in k steps, and consider the first step in which the parser identifies the string h within w such that:

$$\perp^m x < y_1 = y_2 = \dots = y_r > z \perp^n$$

where $h = y_1 y_2 \dots y_r$. Here, x and/or z may be empty, and r may be equal to 1. In any case, the parser must see the relation “ $>$ ” somewhere prior to or at the terminating $\perp^n$. Then, working backward through w from this point, marked by the end string z , it looks for the first “ $<$ ”, marked by the prefix x . In between, the relation “ $=$ ” must apply. If “ $<$ ” and then “ $>$ ” are not found in that order, the string w is not accepted by the parser.

Now the string y so found must also be the right-hand part of some production $A \rightarrow y$ for acceptance; the nonterminal A is also unique because the grammar is uniquely invertible. The “ $=$ ” relation applies among the members of some production right part, and the parser permits a reduction only if y matches a production right part. Hence the parser produces the new string $\perp^m x A z \perp^n$, which it accepts and which, by the inductive hypothesis, is a right-most sentential form or

$$S \Rightarrow^* x A z$$

but $A \rightarrow y$, hence $S \Rightarrow^+ x y z = w$, which shows that w is a right-most sentential form. QED

Theorem 5.4 may also be stated: If a sentence w is not in $L(G)$, then it is not accepted by the precedence parser. This error detection ability is obviously important, since we expect a parser to detect syntax errors as well as to analyze correct sentences. How are errors detected? The parser may encounter a precedence pair with an empty entry in its table, i.e., a pair (x, y) not in any of the three relations. It is also conceivable that it may isolate a string between “ $<$ ” and “ $>$ ” within some sentence that fails to match a production right member. We now give a simple example of a precedence grammar and a sentence which can only be rejected by a failure to match a delimited string to a production right part:

$$\begin{aligned} S &\rightarrow A \mid B \\ A &\rightarrow a \ b \ c \\ B &\rightarrow b \ c \ d \end{aligned}$$

It is easy to show that the $(1, 1)$ precedence table is as follows:

	a	b	c	d	A	B	S	$\perp$
a		=						
b			=					
c				=				>
d								>
A								>
B								>
S								>
$\perp$	<	<			<	<	<	

so that there are no conflicts; the grammar is (1, 1) simple precedence. However, consider the string “abcd”. The parser reaches this configuration:

$$\perp < a = b = c = d > \perp$$

which indicates that “abcd” is the handle, but it fails to match any of the production right parts. Furthermore, the string “abc” probably should have been reduced to A, yet the parser continued past it and failed to detect an error until the $\perp$ was seen. (It is easy to construct grammars similar to this one in which the parser continues an arbitrary number of symbols past an error point).

Summary

Consider a precedence parser for a grammar G , with a table free of conflicts. Then:

1. Given a sentential form in the language $L(G)$, the parser will accept the sentential form, reconstructing a right-most, bottom-up parse.
2. Given some sentence not in $L(G)$, the parser must ultimately fail to accept the sentence; however, it may fail either:
 - (a) by finding a symbol pair not in any of the relations $<$, $=$, or $>$, or
 - (b) by failing to match the left-most string delimited by “ $< \dots >$ ” with the right part of any production.

It is the possibility of failing through case 2(b) that makes a precedence parser less desirable than an $LR(k)$ parser, since it implies that a precedence parser may continue far past a reasonable error point before discovering an error. Nevertheless, much of this “weakness” depends on how one regards a sentence in error. A program with a syntax error should be rejected; no compiler that attempts to “read” the programmer’s mind should be fully

trusted. The important question is not whether an error is detected early or late; it is whether the parser can somehow patch over the error and continue parsing in a reasonable manner, so that there is a fairly good chance of detecting other syntax errors. The patching over process is one form of *error recovery*. Error recovery issues are discussed in chapter 12.

5.2.7. Wirth-Weber Relations

The definitions of a precedence relation given in section 5.2.5 are not particularly useful as a means of obtaining the relations, since they are expressed in terms of the (usually) infinite set of sentential forms rather than the finite set of productions. We may restate the relations in an equivalent, but more useful form, called the *Wirth-Weber (1, 1) precedence relations*:

1. The relation $>$ is always defined on a pair (X, a) , where “ a ” is a terminal symbol, since the symbol immediately to the right of any handle must be terminal. We say that $X > a$, or (X, a) is in $>$ if, for some production $A \rightarrow xBYy$ in P , $B \Rightarrow^+ \dots X$ and $Y \Rightarrow^* a \dots$.
2. A pair of symbols (X, Y) is in $=$ if there exists a production $A \rightarrow \dots XY \dots$ in P . Note that X and Y may be terminal or nonterminal symbols.
3. A pair of symbols (X, Y) is in $<$ if there exists a production $A \rightarrow xXB y$ in P such that $B \Rightarrow^+ Y \dots$.

Equivalence Proof We now show that the Wirth-Weber relations are equivalent to the (1, 1) precedence relations given in section 5.2.5.

Consider the $=$ relation first. Since a handle is always the right member of some production rule, it follows immediately that the (1, 1) precedence relation “ $=$ ” is equivalent to the Wirth-Weber “ $=$ ” relation.

Next consider the $<$ relation, and let (X, Y) be in $<$ by the Wirth-Weber rules; then there exists a right-most derivation

$$S \Rightarrow^* uAv \Rightarrow uxXB yv \Rightarrow^+ uxXYwyv$$

Here $B \Rightarrow^+ Yw$, and Yw is the handle of $uxXYwyv$. Then by the (1, 1) precedence relation (3), (X, Y) must be in $<$.

Now suppose that (X, Y) is in $<$ by the (1, 1) precedence relations. Then there exists a sentential form rhs such that h is the handle, X is the tail symbol of r , and Y the head symbol of h . Now this form has a derivation tree in which X and Y are leaf symbols (they may be nonterminal symbols). The upward paths from X and Y must intersect in some nonterminal symbol A , and A is associated with some production $A \rightarrow w$. Now X must be in w ; that is, A is X 's parent. Otherwise, X is derived from a nonterminal in w , and a handle would have to exist in r , contradicting the assumption that h is the handle in rhs .

Now Y cannot be in w , since the handle must be derived from $Y \dots$ in the A production, not simply be a right part of it. These considerations, put together, imply the Wirth-Weber relation 3, namely, that X must be in a production $A \rightarrow xXB_y$, and $B \Rightarrow^+ Y \dots$

Finally, consider the $>$ relation. Given the Wirth-Weber relation 1, and production $A \rightarrow xBY_y$ such that $B \Rightarrow^+ \dots X$ and $Y \Rightarrow^* a \dots$, then the string $\dots X$ derived from B is a handle at some point in a derivation, and is followed by the terminal symbol " a ". Therefore (X, a) is in the $(1, 1)$ precedence relation. Then suppose that the sentential form rhs exists, where h is the handle, and let " a " be the first symbol in s and X the last symbol in h . Consider the derivation tree for this sentential form and paths up the tree from the leaf nodes X and " a ". These paths must converge first on some nonterminal node A , associated with a production $A \rightarrow xBY_y$, where the X path goes through B and the " a " path goes through (or is) Y . By arguments similar to those given for the $<$ relation, B must derive $\dots X$ in at least one step, otherwise $\dots X = h$ is not a handle. However, Y may be " a " or may derive " $a \dots$ ", which implies the Wirth-Weber relation " $>$ ". QED

Using the Wirth-Weber Relations

The Wirth-Weber relations are valuable in constructing a precedence table since they focus upon the productions and not upon sentential forms. However, we also need some relations that yield derived first and last symbols. We express them as relations in order to develop the simple precedence relations as matrix products.

LAST⁺

$(A \text{ LAST}^+ X)$ if and only if $(A \Rightarrow^+ \dots X)$

This relation defines the set of trailing symbols, terminal and nonterminal, derivable from A in one or more steps.

FIRST^{*}

$(A \text{ FIRST}^* X)$ if and only if $(A \Rightarrow^* X \dots)$

This relation defines the set of leading symbols, terminal or nonterminal, derivable from A in zero or more steps.

TFIRST^{*}

$(A \text{ TFIRST}^* x)$ if and only if $(A \Rightarrow^* x \dots \text{ and } x \in \Sigma)$

Symbol x is a member of the set of leading terminal symbols that can be derived from A , including A itself, if A is terminal.

TRANPOSE

TRANPOSE(M) is the transpose of matrix M, i.e., an element in row i and column j is moved to row j , column i , for all i and j .

Then the Wirth-Weber relations may be concisely defined in terms of the products of certain relations as follows ($\exists$ means *there exists*):

- The $=$ relation is found by inspection of the production set;

$X = Y$ if and only if there exists a production $P \rightarrow uXYv$

- The relation $<$ is equivalent to

$\{(R, S) \mid (\exists A, B)(A \rightarrow \dots RB \dots \text{ and } B \Rightarrow^+ S \dots)\}$, which is equivalent to

$\{(R, S) \mid (\exists B)(R = B) \text{ and } (B \text{ FIRST}^+ S)\}$

Therefore $<$ is equivalent to $(=)(\text{FIRST}^+)$, viewed as a Boolean matrix product of $(=)$ and (FIRST^+) .

- The relation $>$ is

$\{(R, a) \mid (\exists A, B, C)(A \rightarrow \dots BC \dots, B \Rightarrow^+ \dots R, C \Rightarrow^* a, \text{ and } a \in \Sigma)\}$

Now $A \rightarrow \dots BC \dots$ is equivalent to $B=C$, $B \Rightarrow^+ \dots R$ is equivalent to $B \text{ LAST}^+ R$, and $C \Rightarrow^* a \dots$ to $C \text{ TFIRST}^* a$. Therefore

$R > a$ is $(R (\text{TRANPOSE}(\text{LAST}^+))B) \wedge (B=C) \wedge (C \text{ FIRST}^* a)$

hence

$>$ is equivalent to $(\text{TRANPOSE}(\text{LAST}^+))(=)(\text{TFIRST}^*)$,

viewed as a product of Boolean matrices.

The special symbols $\perp$ placed at the beginning and end of a sentence require special attention. The correct treatment is

1. Define a new nonterminal, G , and a new production

$$G \rightarrow \perp S \perp$$

where S is the grammar start symbol. Make G the new start symbol.

2. Include $(\perp, S)$ and $(S, \perp)$ in the $=$ relation.

3. Include $\perp$ in $\text{FIRST}(G)$ and $\perp$ in $\text{LAST}(G)$.

Now when the $<$ and $>$ relations are developed by matrix products, a conflict between $>$ and $=$ will be found in $(S, \perp)$. This conflict is spurious,

and one that the parser will never be concerned about. Delete the $=$ part of it. Also a conflict between $<$ and $=$ will be found in $(\perp, S)$; delete the $=$ part. Any other conflicts are real and must be dealt with. These two arise because we placed $(S, \perp)$ and $(\perp, S)$ in $=$, and we need to do so in order to develop $<$ and $>$ for the other symbols and $\perp$. However, S derives other strings, which means that $(\perp, S)$ will show up in $<$ and $(S, \perp)$ will show up in $>$. The parser must halt when the sentential form $\perp S \perp$ is found.

Example Figure 5.3 gives a worked-out example for a simple grammar, G_2 , containing a binary “ $-$ ” and parenthesizing. The basis grammar G_2 is given in figure 5.1.

Figure 5.3(a) shows the $=$ relation, easily found by inspection of the production set. The pairs $\{E, \perp\}$ and $\{\perp, E\}$ are in the relation because of the first production, $\{E, -\}$ and $\{-, T\}$ are in because of the second production, and $\{(, E)$ and $\{E,)\}$ are in because of the fourth production.

Figure 5.3(b) gives the FIRST relation, easily found from the productions by inspection. Thus $\{G, \perp\}$ is in FIRST because of the first production; hence $G \text{ FIRST } \perp$ is true, etc.

The transitive completion of FIRST is given in figure 5.2(c).

The reflexive transitive completion FIRST^* of FIRST is given in figure 5.3(d). It is simply matrix (c) with the diagonal filled in with 1's.

The relation TFIRST^* , which is all the members of FIRST^* such that the second member of each pair is terminal, is given in (e).

Figure 5.3(f) gives the LAST relation. Thus $\{T,)\}$ is in LAST because of the production $T \rightarrow (E)$, etc.

The transitive completion of LAST is given in (g), and the transpose of LAST^+ is given in (h). This transpose is needed to form the $>$ relation, which is the product of three matrices, figure 5.3(k).

The $<$ relation is developed in figure 5.3(i) as the product of the $=$ relation and the FIRST^+ relation.

Finally, the complete precedence table, which is a summary of tables (a), (i), and (k), is given in (l). Except for the goal symbol row and column, it agrees with figure 5.1. We don't need to include a row and column for the goal symbol since the precedence parser halts on the stack contents $\perp G \perp$.

The $(<, =)$ conflict in $\{\perp, E\}$ and the $(>, =)$ conflict in $\{E, \perp\}$ should be replaced by $<$ and $>$ respectively, removing the spurious $=$ relation membership for these two pairs.

Direct Use of the Wirth-Weber Relations

The use of Boolean multiplication is well suited to a machine computation of an operator precedence table. It is not, however, at all suited to a pencil-and-paper computation. The Wirth-Weber relations may be used to determine an operator precedence table for a small grammar directly. We still

	-	(	)	a	E	T	G	⊥
-					1			
(				1				
)								
a								
E	1		1					1
T								
G								
⊥					1			

(a) =

	-	(	)	a	E	T	G	⊥
-								
(								
)								
a								
E					1	1		
T		1		1				
G								1
⊥							1	

(b) FIRST

	-	(	)	a	E	T	G	⊥
-								
(								
)								
a								
E	1		1	1	1			
T	1		1					
G								1
⊥							1	

(c) FIRST +

	-	(	)	a	E	T	G	⊥
-	1							
(		1						
)			1					
a				1				
E	1		1	1	1			
T	1		1			1		
G							1	1
⊥							1	1

(d) FIRST*

	-	(	)	a	E	T	G	⊥
-	1							
(		1						
)			1					
a				1				
E	1		1					
T	1		1					
G								1
⊥								1

(e) TFIRST*

	-	(	)	a	E	T	G	⊥
-								
(								
)								
a								
E					1			
T		1	1					
G								1
⊥								1

(f) LAST

Figure 5.3. Development of the precedence table for grammar G_1 , using Boolean matrix operations.

	-	(	)	a	E	T	G	⊥
-								
(								
)								
a								
E			1	1		1		
T			1	1				
G								1
⊥								

(g) LAST +

	-	(	)	a	E	T	G	⊥
-								
(								
)					1	1		
a					1	1		
E								
T					1			
G								
⊥							1	

(h) TRANSPOSE (LAST +)

	-	(	)	a	E	T	G	⊥
-		1	1					
(		1	1	1	1			
)								
a								
E								1
T								
G								
⊥		1		1	1	1		

 (i) $< \equiv (=)$ (FIRST +)

	-	(	)	a	E	T	G	⊥
-								
(								
)	1		1					1
a	1		1					1
E								
T	1		1					1
G								
⊥								

 (j) TRANSPOSE (LAST +) ($=$)

	-	(	)	a	E	T	G	⊥
-								
(								
)	1		1					1
a	1		1					1
E								
T								
G	1		1					1
⊥								

 (k) $> \equiv$ (TRANSPOSE (LAST +))
 $=$ (TFIRST*)

	-	(	)	a	E	T	G	⊥
-		<	<	<	=			
(		<	<	<	$\odot$	<		
)	>		>					>
a	>		>					>
E	=		=					$\odot$
T	>		>					>
G								
⊥	<		<	$\odot$	<			

(l) Complete precedence table

Figure 5.3. (cont'd.)

need the functions TFIRST^* , LAST^+ , and LAST^* for each of the nonterminal symbols. Since there cannot be any empty productions, these are particularly easy to determine by inspection.

The $=$ relation is easily found by inspection.

The $<$ relation may be systematically found by finding all the occurrences of any symbol X followed by a nonterminal B in the right member of a production; let such a production be $A \rightarrow xXB$. Then we include all the pairs (X, Y) , where $B \text{ FIRST}^+ Y$, i.e., Y is in the FIRST^+ set of nonterminal B .

The $>$ relation may be systematically determined by finding all occurrences of a nonterminal B followed by any other symbol Y in a production. Then we take every symbol X such that $B \text{ LAST}^+ X$ and every symbol x such that $Y \text{ TFIRST}^* x$ and add (X, x) to $>$.

For example, consider grammar G_2 , and the $>$ relation. We find from the productions that we should consider the pairs:

B		Y
E		$\perp$ from the first production $G \rightarrow \perp E \perp$
E		$-$ from the second production $E \rightarrow E - T$
E		$)$ from the fourth production $T \rightarrow (E)$

The relation $E \text{ LAST}^+ X$ is valid for $X = \{T,), a\}$, and the symbols " x " are just $\{\perp, -,)\}$, hence the $>$ relation must contain the pairs drawn from $\{T,), a\} \times \{\perp, -,)\}$, and these are shown in figure 5.3(l).

Exercises

1. Use the Wirth-Weber relations directly to derive figure 5.1. from grammar G_2 .
2. Construct a $(1, 1)$ precedence table for grammar G_0 , using the Wirth-Weber relations.
3. Write a Pascal program that generates a $(1, 1)$ precedence table, given a grammar in suitable form. Begin by devising a suitable data structure to represent the grammar and other needed tables.

5.2.8. Other Precedence Parsers

A grammar is a *simple precedence* grammar if it is uniquely invertible and its simple precedence table contains no conflicts. As we have seen, even the simple grammar G_1 exhibits a conflict. As a grammar grows in complexity, more conflicts are likely to appear, and they may increase in severity.

The conflicts found in G_1 are between $<$ and $=$. When such conflicts only are found, they create a certain problem in locating the left-most end of a handle, but do not affect the identification of the right-most end. It is often possible to unambiguously fix the handle by some simple rule, such as “match the longest possible production right-hand member” when a conflict is seen. When it is possible to so identify a handle despite a ($<$, $=$) conflict, we say that the grammar is a *weak precedence* grammar.

A conflict with $>$ is much more serious; it means that the precedence table cannot deterministically locate the end of the handle. However, it may be possible to transform the grammar to remove the conflict or to consider a larger (m, n) in the precedence table.

A larger (m, n) than $(1, 1)$ is impractical if the entire precedence table must be based on such strings. The size of an (m, n) table grows very rapidly with m and n , since it must consider a large number of possible combinations of m characters and n characters.

McKeeman, Horning, and Wortman [1970] proposed a secondary precedence table that applies only when the primary $(1, 1)$ table contains a conflict in relation $>$. The secondary table is a $(2, 1)$ table, which they find is sufficient to resolve most of the conflicts remaining in common programming grammars. Since the $(2, 1)$ table need only consider the class of strings associated with the $(1, 1)$ precedence pair found at the conflict, it need not be large. We shall not further consider the interesting question of deriving a secondary table, as the matter is thoroughly discussed in their text. Such a parser as this is called a *mixed strategy precedence* or *MSP parser*.

An *operator precedence* parser is constructed by considering precedence relations among terminal symbols only. The word “operator” arose through the consideration of grammars in which most of the terminals were in fact algebraic operators; operator precedence works rather nicely for them. Many compilers use operator precedence for arithmetic expressions and recursive descent for the other structures.

The construction of an operator precedence parser table is exactly like that for a simple precedence table, except that the nonterminals are ignored within the productions when looking for pairs of symbols. Grammar G_0 is a classic example of an operator precedence grammar. Its operator precedence relations are given in the following table:

	(	a	*	+	)	$\perp$
)			$>$	$>$	$>$	$>$
a			$>$	$>$	$>$	$>$
*	$<$	$<$	$>$	$>$	$>$	$>$
+	$<$	$<$	$<$	$>$	$>$	$>$
(	$<$	$<$	$<$	$<$	$=$	
$\perp$	$<$	$<$	$<$	$<$		

For example, “(” = “)” is obtained from the production $F \rightarrow (E)$, by ignoring E . The relation member “a” > “+” is obtained from the production $E \rightarrow E + T$ and $\text{LAST}(E)$ contains “a”. The precedence relations are generated essentially as before, except that the FIRST^* and LAST^* relations are redefined as follows:

- $X \text{ FIRST}^* a$ if and only if $X \rightarrow^* xay$ and x is empty or consists only of nonterminals.
- $X \text{ LAST}^* a$ if and only if $X \rightarrow^* xay$ and y is empty or consists only of nonterminals.

That is, we examine the sentential forms derivable from X , and ignore leading (in the case of FIRST^*) or trailing (in the case of LAST^*) nonterminals, picking out only the terminal symbols.

It is remarkable that algebraic expressions can be reduced to such a simple set of rules as conveyed by an operator precedence table. Such a table in fact conveys the popular notion of the *strength* or *hierarchy* of operators in a programming language. Precedence is a useful way of explaining to a neophyte programmer the ordering relations among various operators. For example, he may be told in a programming manual that “* has higher precedence than +”. This means that a “*” production will be reduced prior to a “+” production, causing a multiply to be emitted first, whether “*” precedes or follows “+”.

Now consider two operators with equal precedence, say + and −. The “equal” precedence means (in program terms) that the left-most of two operators are applied first, regardless of which kind it is. In parsing terms, the left-most production is reduced first.

In terms of productions, two operators, “@” and “#”, have *equal precedence* if they are part of productions associated with the same left member:

$$\begin{aligned} E &\rightarrow E @ T \\ E &\rightarrow E \# T \\ E &\rightarrow T \end{aligned}$$

On the other hand, an operator “@” has *higher precedence* than “#” if it is derivable from a production containing “#”, e.g.,

$$\begin{aligned} E &\rightarrow E \# T \\ E &\rightarrow T \\ &\vdots \\ T &\rightarrow^* .. @ .. \end{aligned}$$

This production set is such that the T production containing “@” will be reduced prior to the $E \rightarrow E \# T$ production.

In this way, it is possible to design a grammar that contains any desired precedence or ordering relationship among its operators and other functions.

Exercises

1. Verify the operator precedence table given for grammar G_0 .
2. Trace an operator precedence parse in G_0 for the following input strings:

$a + (a * a)$
 $a * a * ((a) + a)$

3. Examine the problem of identifying the handle and detecting syntax errors through failure to identify the handle, for an operator precedence parser.
4. Write an operator precedence parser in Pascal.
5. Sometimes a precedence table is such that we can assign a positive integer $f(t)$ to each token t , where $f(t)$ is such that

$f(a) < f(b)$ if and only if $a < b$
 $f(a) = f(b)$ if and only if $a = b$
 $f(a) > f(b)$ if and only if $a > b$

How useful is f in a parser? Devise an algorithm that determines if function f exists, and apply it to the operator precedence table for G_0 .

6. Design a grammar for a replacement statement language that contains these binary operators:

$+ \quad - \quad * \quad / \quad \uparrow \quad \text{AND} \quad \text{OR} \quad :=$

and parenthesizing. The precedences are

highest AND
 OR
 $\uparrow$
 $* \quad / \quad \{ * \text{ and } / \text{ have equal precedence} \}$
 $+ \quad - \quad \{ + \text{ and } - \text{ have equal precedence} \}$
 lowest $:=$

Also $\uparrow$ and $:=$ are to associate from right to left; all the other operators are to associate from left to right.

7. Consider the following productions in a grammar G:

$$\begin{aligned} S &\rightarrow yAy \mid Cq \\ A &\rightarrow Az \mid B \\ B &\rightarrow tqz \mid B \\ C &\rightarrow Ct \mid v \end{aligned}$$

Is G a simple precedence grammar? If so, show that it satisfies the Wirth-Weber conditions. If not, how does it fail? Can you transform the grammar into an equivalent simple precedence grammar?

8. A string expression $\langle \text{sexp} \rangle$, is defined by the following set of productions:

$$\begin{aligned} \langle \text{sexp} \rangle &::= \langle \text{sexp} \rangle : L / \langle \text{cexp} \rangle \mid \langle \text{cexp} \rangle \\ \langle \text{cexp} \rangle &::= \langle \text{cexp} \rangle * \langle \text{prim} \rangle \mid \langle \text{prim} \rangle \\ \langle \text{prim} \rangle &::= \langle \text{literal} \rangle \mid (\langle \text{sexp} \rangle) \\ \langle \text{literal} \rangle &::= L \langle \text{literal} \rangle \mid \epsilon \end{aligned}$$

L is any letter. This grammar may be transformed into an operator grammar. How? What are the precedence relations among the terminal symbols of the grammar?

9. In many languages, the logical operators AND, OR and NOT and the arithmetic operators “+”, “*”, “/”, “–”, unary “–”, are allowed to be mixed. For example, the assignment

$$a = a \text{ AND } b + c$$

is a valid assignment statement; it means the bit-by-bit AND of the variables a and b is to be interpreted as some number and added to c. Construct a grammar that includes these operators, such that an arithmetic operator has a higher precedence than any logical operation, AND and OR have equal precedence, and AND and OR defer to NOT. Develop the operator precedence table systematically for your grammar and demonstrate that these precedence relations in fact pertain.

5.3. Bibliographical Notes

The idea of using adjacent operators to control a recognizer was introduced intuitively by Perlis [1956]. It eventually became apparent that operator precedence was closely related to the language structure as expressed by its context-free grammar. Paul [1962] essentially solved a class of recognition problems in his thesis; his work did not become known in the U.S. for several years, however. Floyd's paper on operator precedence [1963] then became the first rigorous treatment of the problem of mechanically generating a parser given a context-free grammar. Wirth and Weber (Wirth [1966]) generalized Floyd's ideas into simple and weak precedence.

CHAPTER 6

BOTTOM-UP LR(k) PARSERS

6.1. LR(k) Grammars and Parsers

The LR(k) parsers comprise a family of bottom-up parsers that come as close to ideal left-to-right parsers as is theoretically possible. An LR(k) parser not only identifies a handle but also the production associated with the handle, with no additional decisions required of the rest of the compiler. Furthermore, it takes as much information as it possibly can from the portion of the program preceding and including the handle (the *viable prefix*). The only limitation is that it may examine at most k tokens in the input list past the handle.

An LR(k) parser for a grammar can always be constructed if any deterministic bottom-up parser for the grammar, limited to a k -symbol lookahead, can be constructed. This means that the largest class of grammars are covered by an LR(k) parser. Furthermore, an LR(k) parser covers every top-down LL(k) parser and can accept grammars not accepted by an LL(k) parser.

6.1.1. LR(k) Grammars

Let G be some grammar with start symbol S and consider a right-most derivation of a terminal string w in the grammar:

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w$$

Now consider a typical step in the derivation, as follows:

$$uAv \Rightarrow uxv$$

where $A \rightarrow x$ is the production used in this step, and uxv is one of the w_i or w itself.

We say that G is LR(k) if, for every such derivation and derivation step, the production $A \rightarrow x$ can be inferred by scanning ux and (at most) the first k symbols of v . Note that since A is the right-most nonterminal in uAv , v must be a terminal string.

Given that G is LR(k), we have several useful properties that make possible the development of a deterministic bottom-up parser:

1. The parser will know when to cease scanning a given sentential form uxv , i.e., it can detect the boundary between x and v .
2. The parser will be able to identify the handle x .
3. The parser will be able to uniquely select the production $A \rightarrow x$ that corresponds to the handle and to this sentential form. It happens that a grammar can be LR(k) and yet have several productions $A \rightarrow x$, $B \rightarrow x$, etc. with the same right member.
4. The parser will know when to halt.

As a totally impractical means of constructing a parser for an LR(k) grammar, we might somehow construct a large table that can map every string of the form uxv' , where v' is a k -symbol head of v , to a production $A \rightarrow x$. Unfortunately, for most grammars, the table would have to be of infinite size, since the strings u can be indefinitely long. (However, the strings xv' comprise a finite set.) We need a finite table for a practical compiler.

Knuth [1965] has shown that the set of all viable prefixes of right-most sentential forms can be recognized by a finite state automaton. This FSA can be used as the control machine in an LR(k) parser.

We first define an LR(1) parser. The “1” means that at most one token past the viable prefix will have to be examined in order to make any parsing decision. For most grammars, this token, called a *lookahead token*, is needed only for certain parsing decisions.

There are several different ways of constructing an LR(1) parser (or LR(k) in general), but every such parser operates the same way. The different construction means are classified as SLR(k), SLALR(k), LR(k), and LALR(k). The difference in the parser (if any) appears in the number of states and the size of a set of lookahead tokens when a k -symbol lookahead is required.

Each of the construction methods can be generalized to a k -symbol lookahead, although we shall develop only the case $k = 1$ in depth.

6.1.2. An LR(1) Parser

An LR(1) parser consists of an input list, a stack, and a finite control, essentially as required of any parser for a context-free grammar. The finite control in an LR(1) parser has a fairly large number of states. In a rough sense, the states keep track of vital information in the viable prefix.

Figure 6.1 shows an example finite-state control for grammar G_0 . Notice that this machine contains transitions on both terminal symbols (a , $+$, $*$, $($, $)$, $\perp$), and on nonterminal symbols (E , T , F , G). The parsing principle is quite simple—the finite control recognizes the viable prefix of any right-most sentential form, and falls into a state that uniquely identifies a production exactly when the handle is finished.

Three kinds of states exist in the machine of figure 6.1, *read* states, *lookahead* states, and *apply* states. In a read state, the parser must accept the

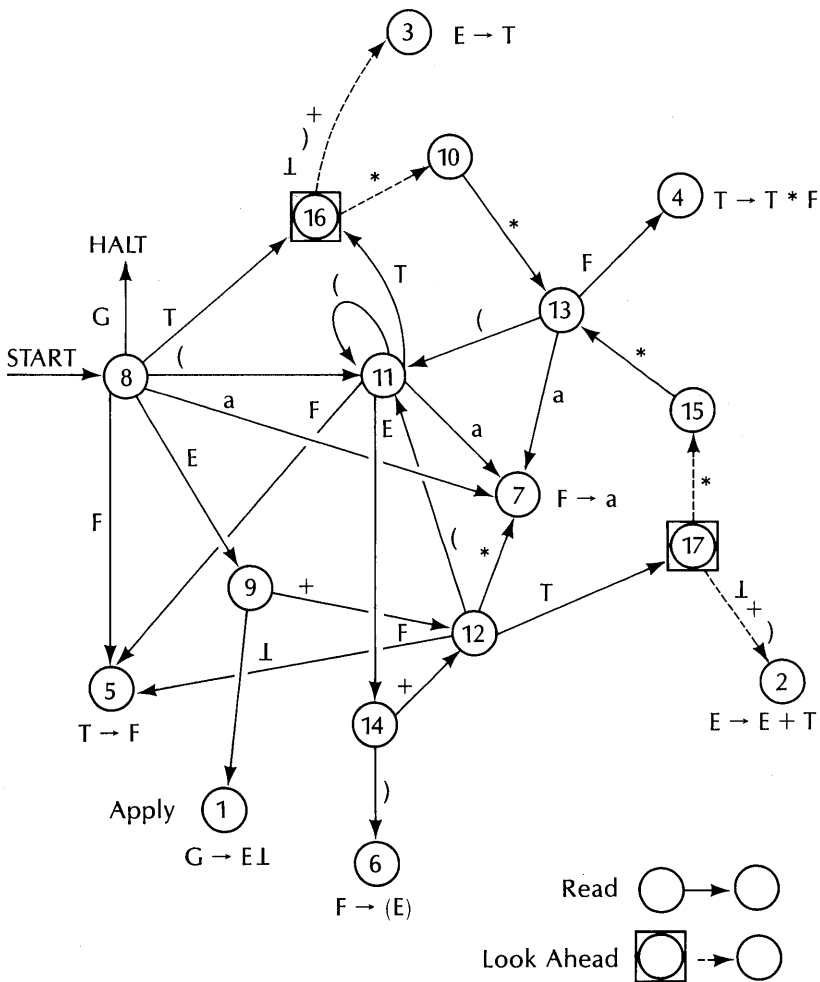


Figure 6.1. Complete LALR(1) parser for grammar G_0 .

next token in the input list, shifting it to its stack, then move to some new state associated with the token. The read states in figure 6.1 are 8, 9, 10, 11, 12, 13, and 14. In a lookahead state, the machine “looks at” the next token, in order to make a state transition, but does not shift it into the stack. The lookahead states in figure 6.1 are 16 and 17. In an apply state, a production is indicated whose right member will be on top of the stack (it is the handle) and should be replaced by the left member of the production. At this point, the original sentential form has been scanned just past its handle, and the state is associated with a production.

The stack will contain the handle and the string preceding the handle upon falling into an apply state. Upon the reduction, the stack followed by the remaining input string comprises the next sentential form subject to a reduction. Therefore, conceptually, we construct a new input list consisting of the stack followed by the remainder of the old input list, and start over again. The apply states in figure 6.1 are numbered 1 through 7 and correspond to the production numbers.

In one cycle of the system, a right-most sentential form xwy is transformed into another one, xAy , such that

$$S \Rightarrow * xAy \Rightarrow xwy \text{ (right-most)}$$

Example. Let us trace the machine of figure 6.1 with a sentence, " $a + (a*a)\perp$ ". ($A\perp$ is added to every sentential form to provide a uniform lookahead and halting symbol). The start state is number 8, and it indicates a transition on "a" to state 7, an apply state. The machine configuration at this point is like this:

stack: a

remainder of input list: $+(a*a)\perp$

State 7 indicates a reduction with the production $F \rightarrow a$, hence the new machine configuration is

stack: F

remainder of input list: $+(a*a)\perp$

These two strings together comprise the next-to-last sentential form in a right-most derivation of " $a + (a*a)\perp$ ", i.e.,

$$G \Rightarrow * F + (a*a)\perp \Rightarrow a + (a*a)\perp$$

One cycle of the machine's operation is now complete. In the next cycle, we have the sentential form " $F + (a*a)\perp$ ", and we start over in state 8. This time, the indicated transition on F is to state 5, another apply state, which then yields the sentential form

$$T + (a*a)\perp$$

upon reduction. Then starting over with this in state 8, we are first led to the lookahead state 16 on symbol T. In this state, we should transfer to state 3, since "+" is a legal symbol along that transition path. The "+" is not shifted into the stack, however, which means that the apply state 3 operates on T and yields the next sentential form:

$$E + (a*a)\perp$$

When we start over again, the machine indicates a move on E to state 9, then on “+” to state 12, then on “(” to state 11, and finally on “a” to state 7. State 7 indicates that the handle is “a” and is to be reduced to F:

$$E + (F*a) \perp$$

is the resulting sentential form.

Figure 6.2 summarizes the complete parsing process on the string “a + (a*a) $\perp$ ”. (We have numbered the variables “a” to distinguish them; the machine makes no such distinction.) It can be seen from the apply column in figure 6.2, which gives the indicated productions to be applied in each cycle, that the machine reconstructs a right-most parse of the sentence, in reverse order, but of course does so through a left-to-right scan.

The parsing process ends upon a move from the start state 8 to *halt*, which occurs on the goal symbol G. It may sometimes happen that a lookahead is required on a goal symbol move; but there must always be some halt state that indicates that the parse is successful.

The parser also rejects every invalid sentence, i.e., a sentence not in the language $L(G_0)$. Furthermore, it does so on the very first illegal symbol. Such an error detection must occur in a lookahead or a read state, never in an apply state, since these must accept or reject the next input symbol from the input list. When the parser is in some read or lookahead state, it can only accept a certain subset of the language’s alphabet. For example, if in state 11, the parser may only accept one of the terminal tokens {a, (,)} next in the input list. (The nonterminal tokens don’t count for error detection; why not?) Failure to see one of these must be a syntax error and may be so immediately reported.

The problem of dealing with an error will be discussed at greater depth in chapter 12. For now, we discuss a number of reductions on the parser operations and table sizes.

Parser Reduction

We first discuss a reduction in machine operations, making use of a push-down stack to achieve this. The stack will contain state numbers, which correspond to the symbols in a viable prefix of a sentential form.

Note first from figure 6.1 that every state has associated with it exactly one incoming symbol. For example, state 5 is associated with the *in-symbol* F, state 11 with “(”, etc. This property is true for every LR(1) incremental parser, as we shall see upon defining the construction process. (It is not true in general for an arbitrary finite state machine.) We can therefore replace the symbols in the viable prefix of some sentential form by states with which they are associated (however, note that a given symbol may be the in-symbol of more than one state, e.g., E is an in-symbol for states 9 and 14). The list of states is effectively the states passed through during the scan of the sentential

Input string = $a_1 + (a_2 * a_3) \perp$

States	Apply	Stack	Input
8, 7	$F \rightarrow a_1$	F	$+ (a_2 * a_3) \perp$
8, 5	$T \rightarrow F$	T	$+ (a_2 * a_3) \perp$
8, 16, 3	$E \rightarrow T$	E	$+ (a_2 * a_3) \perp$
8, 9, 12, 11, 7	$F \rightarrow a_2$	E + (F	$* a_3) \perp$
8, 9, 12, 11, 5	$T \rightarrow F$	E + (T	$* a_3) \perp$
8, 9, 12, 11, 16, 10, 13, 7	$F \rightarrow a_3$	E + (T * F	) $\perp$
8, 9, 12, 11, 16, 10, 13, 4	$T \rightarrow T * F$	E + (T	) $\perp$
8, 9, 12, 11, 16, 3	$E \rightarrow T$	E + (E	) $\perp$
8, 9, 12, 11, 14, 6	$F \rightarrow (E)$	E + F	$\perp$
8, 9, 12, 5	$T \rightarrow F$	E + T	$\perp$
8, 9, 12, 17, 2	$E \rightarrow E + T$	E	$\perp$
8, 9, 1	$G \rightarrow E \perp$	G	
8, HALT		HALT	

Figure 6.2. Trace of LALR(1) parser of figure 6.1 for input $a + (a * a) \perp$.

form. For example, consider the sentential form " $E + ((a))$ ", in which the handle is " a ". Upon scanning " a ", the stack will contain states 8, 9, 12, 11, and 7.

The right parentheses are not associated with states, since they are not yet part of a viable prefix. Upon reducing the handle, " a ", to F , since this is indicated by state 7, we have the stack 8, 9, 12, 11, 5, which corresponds to the sentential form " $E + ((F))$ ". We may view the correspondence of stack states and the viable prefix symbols as follows:

$$E_8 +_9 (_{12} (_{11} F_5))$$

Note that the state number (11) just preceding the handle (F) has not changed; it cannot, since it is associated with a symbol in the viable prefix preceding the handle. Clearly, we do not need to rescan the entire sentential form upon each reduction; it is only necessary to pick up at the state just preceding the handle. The states associated with the handle are popped, a new nonterminal is pushed, and a transition is made that depends on the state just below the handle.

The apply operation suggests another reduction. One nonterminal token is pushed in an apply, then the next state is determined by the (former) top-of-stack state and this nonterminal. We may therefore construct a table that maps a production number and the top-of-stack state (found after popping the number of states required by the apply) into a next state. With

such a table and operation, we no longer need any of the nonterminal moves in the read table.

Representation of the Parser as Tables

An efficient representation for an LR(1) parser consists of four tables and an interpreter. Three of these are shown in figure 6.3; the fourth table is a push-state table, and is unnecessary for the parser of figure 6.1.

The convention for this parser is that the stack contains every state except the “present” state.

The READ table yields a next state number, given a current state and a next symbol. The rule for entering the READ table is that the state must lie between 8 and 15, inclusive; then the next symbol is read and scanned; then the old state is pushed onto the stack. Note that we push the state we just left, not the one entered. A blank entry is a syntax error; errors are detected on the left-most symbol that cannot be a part of any sentential form.

The LOOKAHEAD table applies to the lookahead states 16 and 17; it is organized in a manner similar to the READ table, except that it is not necessary to list all the possible symbols involved in a given lookahead; hence the “else” column. We enter the table with a lookahead state and some next symbol in the input list. If the symbol is not found in the main body, then the “else” state is taken. The lookahead symbol is not scanned; eventually it will be scanned in a read state, at which time a possible syntax error will be detected.

The APPLY-GOTO table is actually two tables, entered by the state number 1 to 7, figure 6.3. The state number will always also be the production number. The second column in figure 6.3 (*pop*) is the number of symbols to pop from the stack before pushing another state number; this number is one less than the number of symbols in the right part of a production, except for an empty production, for which the pop number would be zero.

The GOTO portion of the APPLY-GOTO table contains the nonterminal transitions in the LR(1) parser machine of figure 6.1. Recall that upon identifying the handle (on top of the stack), the apply operation pops the handle and pushes a nonterminal symbol. Since we wish the stack to contain states, not symbols, and this is the only operation in which nonterminal symbols are pushed, we instead examine the state on the stack top just after the handle is popped. There must be a transition on this state and on the pushed nonterminal to some other state. The GOTO table contains the next state associated with the production state and the top-of-stack state. For example, suppose the machine falls into state 2. Then the POP column says that 2 states are to be popped. The top-of-stack state may then be 11 or some other state. If 11, then we go to state 14; if some other state, then we go to state 9. State 11 is left on the stack.

The GOTO table is obtained directly from the nonterminal transitions in

Read table for Go

1. Read
2. Push old state

State	Symbol					
	L	+	*	(	)	a
8				11		7
9	1	12				
10			13			
11				11		7
12				11		7
13				11		7
14		12			6	
15			13			

Lookahead Table

1. Look ahead

State	L	+	*	(	)	a	Else
16			10				3
17			15				2

Apply – Goto Table

1. Apply production
2. Pop stack
3. Go to

State	Pop	Go to				Production
		11	12	13	Else	
1	1				halt	$G \rightarrow E.L$
2	2	14			9	$E \rightarrow E + T$
3	0	14			9	$E \rightarrow T$
4	2		17		16	$T \rightarrow T * F$
5	0		17		16	$T \rightarrow F$
6	2			4	5	$F \rightarrow (E)$
7	0			4	5	$F \rightarrow a$

Figure 6.3. LALR(1) parser in table form.

figure 6.1 and reduced by lumping together the most popular top-of-stack state into an ELSE column, for each apply state. For example, states 2 and 3 both call for pushing the nonterminal E, therefore the E transitions in figure 6.1 belong in the GOTO table; these are

8 to 9,
11 to 14

The “11 to 14” transition is given directly, while the “8 to 9” transition is in the ELSE column. Similarly, for the apply states 6 and 7, the transitions on F are wanted; these are

8 to 5,
12 to 5,
11 to 5,
13 to 4

Since the most popular destination state is 5, the top-of-stack states 8, 11, and 12 are included in the ELSE column, while the “13 to 4” transition is specifically included.

In summary, the APPLY-GOTO table is entered with a state number that corresponds to a production. The right part of that production is on the stack top, and some semantic action may be taken at this point in the parse. After the semantic operations are complete, the stack is popped off the number of symbols in the POP column, and the stack top state P is used to enter the GOTO table for the current production, to yield a new state.

If the grammar contains any empty productions, a PUSH table is needed. Consider an APPLY state in the incremental parser for an empty production, and recall that the current state is not on the stack top (the previous state is). We therefore need another table called a PUSH table that maps an empty production apply state (the current state) to a next state. The stack is unaffected.

Example Parse with the Tables

Let us parse the string “ $a*(a+a)\perp$ ” with the aid of figure 6.3. The start state is 8. Since 8 is a read state, we enter the READ table and scan symbol “a” in the input list; the table says the next state is 7. (See the first row in figure 6.4.) State 8 is pushed onto the stack.

State 7 is an APPLY state. Production 7 is $F \rightarrow a$. Upon entering the APPLY table, it indicates a pop of zero states; the stack top is 8, hence the GOTO table indicates the next state is 5; the stack contains only 8 at this point.

State 5 is also an APPLY state, with a zero pop, production $T \rightarrow F$, and the next state is 16.

State 16 is a lookahead state. The next symbol in the input list is “*”, so the LOOKAHEAD table indicates the next state is 10. The stack still contains only 8, and only the first symbol, “a”, has been scanned in the input list.

The remainder of the parsing operation is given in figure 6.4. The parsing ends upon entering state 1, which is associated with the goal production $G \rightarrow E \perp$. State 8 remains on the stack at this point.

Further Table Reduction

The tables in figure 6.3 may be reduced by using the sparse matrix techniques discussed in chapter 2, and exploiting identical rows in the original tables. For example, the READ table of figure 6.3 may be compressed to the following tables:

Trace of $a * (a + a) \perp$						
State	Operation	Input	Pop	Push	Stack	Goto
8	R	a		8	8	7
7	A		0		8	5
5	A		0		8	16
16	L	*			8	10
10	R	*		10	8, 10	13
13	R	(		13	8, 10, 13	11
11	R	a		11	8, 10, 13, 11	7
7	A		0		"	5
5	A		0		"	16
16	L	+			"	3
3	A		0		"	14
14	R	+		14	8, 10, 13, 11, 14	12
12	R	a		12	8, 10, 13, 11, 14, 12	7
7	A		0		"	5
5	A		0		"	17
17	L	)			"	2
2	A		2		8, 10, 13, 11	14
14	R	)		14	8, 10, 13, 11, 14	6
6	A		2		8, 10, 13	4
4	A		2		8	16
16	L	$\perp$			8	3
3	A		0		8	9
9	R	$\perp$		9	8, 9	1
1	A		1		8	halt

Figure 6.4. Trace of LALR(1) parser tables of figure 6.3 for input $a*(a+a)\perp$.

S	N	Size
8	1	2
9	3	2
10	5	1
11	1	2
12	1	2
13	1	2
14	6	2
15	5	1

N	Symbol	Destination
1	(	11
2	a	7
3	$\perp$	1
4	+	12
5	*	13
6	+	12
7	)	6

For example, given read state 14, N is 6 and “size” is 2; the legal symbols and destination states are therefore in the second table starting at index N=6, running for 2 items; on “+” the next state is 12 and on “)” the next state is 6. Note that we have also compressed the similar rows 8, 11, 12, and 13 in the READ table into rows 1 and 2 of the reduced table. This kind of compression implies that the N values in the first table will not necessarily be monotonic increasing; we therefore also need the “size” column.

The lookahead and apply tables may similarly be compressed.

Exercises

1. Trace the parser of figure 6.1 on the strings

(a + ((a * a))) $\perp$
a * a * a $\perp$
a * * a $\perp$
a + a) $\perp$

2. Trace the parser of figure 6.3 on the strings of exercise 1.
3. Write an LR(1) parser in Pascal that interprets a set of tables similar to those in figure 6.3. Make it “semantic” driven, so that each call on a

parser procedure returns a production number; it in turn calls a scanner that returns the next token on each call.

6.1.3. LR(0) Parser Construction

We shall now describe a construction algorithm for an LR(0) machine. Such a machine will, in the absence of conflicts, parse sentences without ever requiring a lookahead. It will become the basis for the SLR(k) and SLALR(k) parsers.

The construction of an LR(k) machine will be defined in section 6.2. An alternative construction method for an LR(k) machine is described in section 6.3.

In this construction, each machine state is associated with a set of *items*, where an item is a production carrying a position marker. The rules for constructing the item-set corresponding to some state, and for starting new states, are reasonably simple and lend themselves to a machine implementation. The resulting parser machine is also in minimal form when the process is complete.

Recall that a viable prefix is a prefix of some right-most sentential form that includes no symbols past the handle of the form. We will place much emphasis on a viable prefix for one simple reason: the LR parser finite state machine recognizes viable prefixes, and at the same time a viable prefix is connected with a sentential form. It therefore serves as a kind of conceptual link between derivations (which involve sentential forms) and the LR parsing automaton.

Now consider an arbitrary state P in the complete LR machine, and the set (usually infinite in size) of viable prefixes associated with that state P . A viable prefix w is associated with state P if and only if the machine accepts w upon falling into state P . It should be clear that a given viable prefix can be associated with only one state, because we demand that the machine be deterministic. (We have not yet shown that it is finite.)

We say that two viable prefixes belong to the same LR equivalence class if they are associated with the same state.

The association of viable prefixes with states is an interesting thought, but not very useful, since each state may have an infinite number of viable prefixes associated with it. For example, all of the following viable prefixes in grammar G_0 are associated with state 12:

E+
 (E+
 ((E+
 T*(E+
 T*((E+
 (E+(E+

It is easy to find many more. We need only trace through the states starting at state 8 and ending at state 12. The loop on “(” in state 11 can be traversed any number of times, yielding another left parenthesis each time, as can the loop through 12-11-14. We obviously need to find some finite set that can be associated with each state.

Such a finite set is a set of items. An *item* is a marked production enclosed in brackets, i.e.,

$$[A \rightarrow x \cdot y]$$

is an item if $A \rightarrow xy$ is a production in G . Here, either x or y or both may be empty.

An item $[A \rightarrow x \cdot y]$ is said to be *valid* for some viable prefix ux if and only if some right-most derivation

$$S \Rightarrow^* uAv \Rightarrow uxyv$$

exists. Note that xy is the handle of the sentential form $uxyv$, and that v is a terminal string. The mark “.” in an item associated with some viable prefix indicates that this point in the production marks the end of the viable prefix.

There are in general many items valid for a given viable prefix, although the number must be finite. There can only be a finite number of items altogether, since they consist of a finite number of mark positions in a finite number of productions.

Now consider some state P in the finite state control for an LR parser. State P is associated with some set of viable prefixes, and therefore with some finite set of items valid for those viable prefixes. We propose to identify and distinguish the states in the finite control of the parser by constructing the set of valid items associated with each of the states.

For example, consider the machine of figure 6.1, and state 12. A list of some viable prefixes associated with state 12 is given above. We now construct a set of valid items associated with state 12.

Because state 12 is entered upon scanning “+”, the item $[E \rightarrow E + \cdot T]$ must be associated with state 12. Although this is the only possible item in which the mark can follow “+”, as required by the in-symbol of state 12, this is not the only item associated with state 12. The following are, too:

$$\begin{aligned} [T &\rightarrow \cdot F] \\ [T &\rightarrow \cdot T * F] \\ [F &\rightarrow \cdot (E)] \\ [F &\rightarrow \cdot a] \end{aligned}$$

To show this association, note that the viable prefix

$$E +$$

is part of a sentential form

$$E + T$$

with the derivation

$$E \Rightarrow E + T \Rightarrow E + F$$

and therefore the item $[T \rightarrow .F]$ is valid for the viable prefix “E+”. (From the definition given earlier, in this instance we have $S = E$, $u = E+$, $x = \epsilon$, $y = F$, $v = \epsilon$).

Similarly, the derivation

$$E \Rightarrow E + T \Rightarrow E + T * F$$

shows that the item $[T \rightarrow .T * F]$ is valid for the viable prefix “E+”.

Item-Set Construction

We now see how a finite set of items and a (possibly) infinite set of viable prefixes may be associated with each state in the LR control automaton. We next need to develop rules for building the item-sets and from them, the states and transitions of the LR automaton. The following rules do just that. We shall in due time show that they are correct and make each aspect of them apparent.

The three construction rules for an LR(0) control automaton are called the *start*, the *completion*, and the *read* operations, respectively, and are as follows:

The Start Operation. If S is the start symbol of the grammar, and $S \rightarrow w$ is some production, then item $[S \rightarrow .w]$ is associated with the start state.

This operation is needed to get the construction process started. The other three assume that some states and items associated with the states already exist. The start state may eventually have several items in it.

The Completion Operation. If $[A \rightarrow x . Xy]$ is an item in some state P , then every item of the form $[X \rightarrow .z]$ must be included in state P . Note that X must be a nonterminal symbol, and this rule must be repeated until no more new items can be added to the state.

The Read Operation. Let X be a terminal or nonterminal symbol in an item $[A \rightarrow x.Xy]$ associated with some state P . Then $[A \rightarrow xX.y]$ is associated with a state Q (possibly the same as P), and a transition

$$P \text{ to } Q \text{ on symbol } X$$

exists.

We call this a *read* operation, since it becomes a READ action in the final parser if X is a terminal symbol. When X is a nonterminal symbol, we have seen that a transition on X becomes part of the APPLY action, following the replacement of the handle on the stack top by that nonterminal symbol.

Parser Construction

These three operations are all that are needed to generate a finite control for an LR(0) parser, such as the one in figure 6.1, except for lookahead states. We shall see why lookahead states are necessary and how they are constructed shortly. For now, let us summarize the construction of the finite state system as implied by the three operations just given.

1. Give the start state a number, and use the start operation to put one item into it. Then use the completion operation repeatedly if necessary, to get more items into this state. In completing a state, we look for a nonterminal symbol X that follows the mark “.”, and then add items of the form $[X \rightarrow .w]$ to the state, where $X \rightarrow w$ is a production. Eventually, this completion operation has to end.

2. We now have one state, consisting of a set of items. It will not be any different in principle than any other state constructed by this process, so we consider it a general state.

3. Use the read operation to start one or more new states, based on the present state. The idea is to look for items of the form $[A \rightarrow x.Xy]$, i.e., items in which some symbol X follows the mark, then build a new state from the item $[A \rightarrow x.X.y]$, i.e., the mark “moved past” the symbol X . This new state incidentally must also contain all the other items from the old state in which this symbol X follows the mark. For example, if the old state contains the two items

$$\begin{aligned} [E \rightarrow E+.T] \\ [E \rightarrow .T] \end{aligned}$$

(among others), then the new state must contain (at least) the items

$$\begin{aligned} [E \rightarrow E+T.] \\ [E \rightarrow T.] \end{aligned}$$

Let the old state be P and the new state be Q . Then we know that the finite state machine in the LR parser will have the transition P to Q on X . This fact explains why we carry over into state Q all the items in which X follows the mark.

The “new” state may in fact be exactly like some other state previously constructed. We can tell whether two states are equivalent by examining the list of items associated with them. If the two lists are identical, when completed, then the two states must be equivalent. In this way, the state-building process must eventually terminate, since there can only be a finite number of distinguishable sets of items. Eventually, we have to construct a state that is identical to some state previously constructed.

It is not necessary to compare all the items in two states to judge their equivalence. We need compare only the *core* items; these are the items that are initially placed in a state through the read or start operation. Usually, they have some symbol preceding the mark, but they may not if empty productions exist in the grammar.

4. Complete the new state started in step 3 by applying the completion operation repeatedly.

5. Repeat steps 3 and 4 until no more new states are obtained.

An Example. Let us apply the construction to grammar G_0 ; the yield should be the finite-state machine in figure 6.1. We shall indulge in a bit of sleight-of-hand in assigning numbers to the states, since we know in advance that the first eight states are associated with completed items for the eight productions in G_0 . There is no reason to number the states one particular way or another, except that we already have introduced the finite control for the parser (figure 6.1), and it is less confusing to adopt the state numbers used there.

The productions of G_0 are:

1. $G \rightarrow E \perp$
2. $E \rightarrow E + T$
3. $E \rightarrow T$
4. $T \rightarrow T * F$
5. $T \rightarrow F$
6. $F \rightarrow (E)$
7. $F \rightarrow a$

The production numbers 1 to 7 will be apply state numbers. We assign the next number, 8, to the start state, which initially contains the item $[G \rightarrow .E \perp]$. Now this item can be completed, by adding to state 8 all the items associated with E productions: $[E \rightarrow .E + T]$, $[E \rightarrow .T]$. The second of these in turn leads to a further completion operation, bringing the items $[T \rightarrow .T * F]$ and $[T \rightarrow .F]$ into state 8. Finally, the last item calls for a completion with the items $[F \rightarrow .(E)]$ and $[F \rightarrow .a]$. This operation yields the set of items in state 8, as follows:

8. $[G \rightarrow .E \perp]$
 $[E \rightarrow .E + T]$
 $[E \rightarrow .T]$
 $[T \rightarrow .T * F]$
 $[T \rightarrow .F]$
 $[F \rightarrow .(E)]$
 $[F \rightarrow .a]$

We now apply the read operation to these items. The first two items, with E following the mark, yield a new state, 9, and a transition (8 to 9 on E). The next two items yield state 10, and a transition (8 to 10 on T). The next item ($[T \rightarrow .F]$) yields state 5 and a transition (8 to 5 on F). Finally the last two items yield states 11 and 7, with transitions on "(" and "a", respectively. Each of these states must be worked on by the completion and read rules, yielding new states, etc. Let us therefore continue with state 9.

State 9 arises from the two items $[G \rightarrow .E\perp]$ and $[E \rightarrow .E+T]$. The read operation requires moving the mark past the E symbol, hence state 9 is initialized with the core set

9. $[G \rightarrow E.\perp]$
 $[E \rightarrow E.+T]$

This set is complete, since there are no nonterminal symbols following a mark in either item. The read operation applied to this set yields states 1 (for the first item), with the transition (9 to 1 on $\perp$), and state 12 (for the second item), with the transition (9 to 12 on "+").

State 1 therefore stems from the first item in 9 and looks like this:

1. $[G \rightarrow E.\perp.]$

No completion or read operations are possible. This kind of item, with the mark at the end of the production, is called a *completed* item and is normally associated with a reduction, or apply state.

Another apply state obtained from state 8 is state 7:

7. $[F \rightarrow a.]$

We next show the complete state set when the completion and read operations are carried out to the bitter end. We have added the state transitions to the right of the production, in the form (transition symbol, next state):

- | | |
|------------------------------|----------|
| 8. $[G \rightarrow .E\perp]$ | (E, 9) |
| $[E \rightarrow .E+T]$ | (E, 9) |
| $[E \rightarrow .T]$ | (T, 10) |
| $[T \rightarrow .T*F]$ | (T, 10) |
| $[T \rightarrow .F]$ | (F, 5) |
| $[F \rightarrow .(E)]$ | ((, 11) |
| $[F \rightarrow .a]$ | (a, 7) |

9. $[G \rightarrow E.\perp]$ ($\perp$, 1)
 $[E \rightarrow E.+T]$ (+, 12)
10. $[E \rightarrow T.]$
 $[T \rightarrow T.*F]$ (*, 13)
11. $[F \rightarrow (.E)]$ (E, 14)
 $[E \rightarrow .E+T]$ (E, 14)
 $[E \rightarrow .T]$ (T, 10)
 $[T \rightarrow .T*F]$ (T, 10)
 $[T \rightarrow .F]$ (F, 5)
 $[F \rightarrow .(E)]$ ((, 11)
 $[F \rightarrow .a]$ (a, 7)
12. $[E \rightarrow E+.T]$ (T, 15)
 $[T \rightarrow .T*F]$ (T, 15)
 $[T \rightarrow .F]$ (F, 5)
 $[F \rightarrow .(E)]$ ((, 11)
 $[F \rightarrow .a]$ (a, 7)
13. $[T \rightarrow T*.F]$ (F, 4)
 $[F \rightarrow .(E)]$ ((, 11)
 $[F \rightarrow .a]$ (a, 7)
14. $[F \rightarrow (E.)]$ (), 6)
 $[E \rightarrow E.+T]$ (+, 12)
15. $[E \rightarrow E+T.]$
 $[T \rightarrow T.*F]$ (*, 13)

The first seven states correspond to the set of completed productions:

1. $[G \rightarrow E\perp.]$
2. $[E \rightarrow E+T.]$
3. $[E \rightarrow T.]$
4. $[T \rightarrow T*F.]$
5. $[T \rightarrow F.]$
6. $[F \rightarrow (E).]$
7. $[F \rightarrow a.]$

We now have enough of the finite state controller for a comparison with figure 6.1. Only the state transitions and an indication of the parse states need survive.

Unfortunately, we have a problem with this grammar. Although we would like to assign state 2 to the item $[E \rightarrow E + T.]$, we see that it in fact appears with another item in state 15. Similarly, the item $[E \rightarrow T.]$ should be alone in state 3, but appears in state 10 with another non-completed item, $[T \rightarrow T.*F]$. What are we to make of this situation?

Consider state 10, which contains the item $[E \rightarrow T.]$ and the item $[T \rightarrow T.*F]$. When the finite control reaches this state on some sentential form, there may be two possible moves. The completed item $[E \rightarrow T.]$ essentially says that the parser should reduce the T on the stack top to E . On the other hand, the item $[T \rightarrow T.*F]$ essentially says, "Don't reduce the stack top, shift token $*$ (if seen) into the stack". For example, the sentential form " $T*a$ " leads to just such an indecision. We cannot have it both ways, and we don't want the parser to be nondeterministic, so something must be done to make a deterministic decision at this point.

Inadequate States

State 10 is called an *inadequate* or *inconsistent* state. In general an inadequate state is any state containing both a completed item (of the form $[A \rightarrow w.]$) and any other item. Such a state represents a conflict in a parsing decision, in the same way that a conflict in an operator precedence table represents a potential difficulty in parsing some string. The conflict arises from the grammar. There are grammars for which no LR conflict arises, and others that produce one or more conflicts.

There is no easier way to determine in advance of an item-set construction whether a given grammar will create a conflict. We must simply forge ahead into constructing the state sets and discover the conflicts as they arise. The resolution of inadequate states will be discussed in section 6.1.4.

If the LR states contain no inadequate states, the grammar is said to be $LR(0)$. We are then also sure that the grammar is unambiguous. We shall not prove this result, but it should be apparent from the deterministic character of the parsing machine that results from the state set construction. Of course, we have not yet shown that the parsing machine accepts exactly those sentential forms in the source grammar. We shall prove this assertion next.

Exercise

1. Construct the LR state sets for the following grammar G_3 :

```

prog → block
block → BEGIN decls stmts END
block → stmt
decls → ε
decls → decls D ;
stmts → stmts ; stmt
stmts → stmt
stmt → block
stmt → S

```

Identify its inadequate states. Check the parser by tracing some strings in $L(G_3)$ and some strings not in $L(G_3)$.

Construction Correctness Proof

A proof of correctness of the parser system defined above will establish that the finite state machine constructed (hereafter simply called “the machine”) recognizes exactly the class of viable prefixes of right-most sentential forms. The proof is in two parts. We first show that the items within any state are exactly consistent with the class of viable prefixes for that state. Then we can show that the machine recognizes viable prefixes exactly.

The machine may or may not contain inadequate states. The proofs essentially deal only with read and apply states, and even these are not distinguished as such. We merely consider a machine that can accept a left-most sentential form up to, but not past, its handle, i.e., a viable prefix.

We need a few definitions first.

Recall that an item $[A \rightarrow x \cdot y]$ is said to be *valid* for some viable prefix ux if and only if some right-most derivation

$$S \Rightarrow^* uAv \Rightarrow uxyv$$

exists.

A string w is said to be *associated with* or *valid for* some state P in the machine if and only if the machine falls into state P upon scanning w .

An item is said to be *valid* for some state P in the machine if and only if it is valid for some viable prefix w associated with state P .

Now we may state and prove the first part of the construction correctness.

Lemma 5.2. Every state P contains all and only the items valid for P .

Proof: “Only the” Part. Let $[A \rightarrow x \cdot y]$ be in P through the construction process, and assume that it was placed in P through $k + 1$ steps of the process. We assume the lemma true for $k' \leq k$ steps.

If $x = \epsilon$, and $P \neq S$, then this item got into P through a completion operation through some other item, $[B \rightarrow .Az]$, also in P . By the inductive hypothesis, this item is valid for P , hence there must be a viable prefix u and a derivation

$$S \Rightarrow^* uBv \Rightarrow uAzv \Rightarrow uyzv$$

Therefore $[A \rightarrow .y]$ is also valid for P since viable prefix u is.

If $x = \epsilon$ and $P = S$, then k may be 1, so that we have item $[S \rightarrow .y]$, which is clearly valid for the start state. For $k > 1$ and $P = S$, we have item $[A \rightarrow .y]$, which is valid for the start state by an argument similar to that in the preceding paragraph.

For nonempty x , the item $[A \rightarrow x.y]$ has the form $[A \rightarrow rX.y]$, where $|X| = 1$. It can only be in state P because an item $[A \rightarrow r.Xy]$ exists in some other state R , with a transition (R to P on X). Now $[A \rightarrow r.Xy]$ is valid for some viable prefix ur , valid for R , by the inductive hypothesis, through a derivation

$$S \Rightarrow^* uAz \Rightarrow urXyz$$

But this also shows that $[A \rightarrow rX.y]$ is valid for the viable prefix urX . Finally, if ur is valid for state R , and a transition (R to P on X) exists, then urX is valid for P . Therefore $[A \rightarrow rX.y]$ is valid for P . QED

Proof: "All the" Part. Consider any viable prefix w valid for state P . We use induction on the length of w .

For $|w| = 0$, P must be the start state S_0 . (There are no empty moves from S_0 to any other state). But S_0 contains at least one item $[S_0 \rightarrow .u]$ which is valid for the empty viable prefix.

Now let $|w| = k + 1$, and assume the lemma true for all strings w of length k or less. String w has the form

$$w = xX$$

where $|X| = 1$ and X is in $(N \cup \Sigma)$. With w valid for P , we have the derivation

$$S \Rightarrow^* wy = xXy$$

Consider the step in which X was introduced through a production:

$$S \Rightarrow^* rAz \Rightarrow rsXtz \Rightarrow^* rsXy$$

where $rs = x$, $tz \Rightarrow^* y$, and $A \rightarrow sXt$. This derivation shows that $[A \rightarrow s.Xt]$ is valid for $w = xX = rsX$, hence for P . Is this item in P ? Note that $[A \rightarrow s.Xt]$ is valid for viable prefix x ; since xX is valid for P , then there must be a state R for which x is valid, with a transition (R to P on X). However, the inductive

hypothesis implies $[A \rightarrow s.Xt]$ must be in R , therefore by the construction, $[A \rightarrow sX.t]$ is in P . We have shown that every state contains all the items valid for it. QED

The Main Theorem

We now turn to the principal theorem:

Theorem 5.5. The machine M (constructed as above) recognizes exactly the class of viable prefixes of grammar G .

An immediate consequence of this theorem is that M will serve as a (possibly) nondeterministic bottom-up parser for sentences in G . It will be deterministic if there are no inadequate states; in that case, the handle of any sentential form will be identified by falling into an apply state containing a single completed item $[A \rightarrow w.]$. If there are inadequate states, then each of these represents a nondeterminism which must be resolved by lookahead methods to be developed later.

Proof: (M recognizes every viable prefix in G .) First, let w be some viable prefix in G ; then we have

$$S \Rightarrow^* wv \text{ (right-most)}$$

Let v be terminal, without loss of generality. We prove by induction on the number of derivation steps that M recognizes w .

For zero steps, the start state clearly recognizes the class of empty viable prefixes. Therefore, consider a $(k+1)$ -step derivation,

$$S \Rightarrow^* uAv' \Rightarrow uxv'$$

where A is such that w is some prefix of ux . If w is a prefix of u , then by the inductive hypothesis, M recognizes u and therefore w . Hence let w be a prefix of ux but not u . Let $x = X_1X_2 \dots X_ny$, where each of the X_i are of length 1, such that

$$w = uX_1X_2 \dots X_m$$

and $m \leq n$. We then have

$$S \Rightarrow^* uAv' \Rightarrow uX_1X_2 \dots X_ny v'$$

Now u is accepted by M by falling into some state P_0 for which u is valid, by induction. The derivation shows that $[A \rightarrow .X_1X_2 \dots X_ny]$ is valid for u , and by lemma 5.2, must be in P_0 . Then by the construction process, there must be a sequence of states $P_1, P_2, \dots, P_n$ associated with items and viable prefixes as follows:

- P_1 with $[A \rightarrow X_1.X_2X_3 \dots X_n.y]$ and uX_1 .
- P_2 with $[A \rightarrow X_1X_2.X_3 \dots X_n.y]$ and uX_1X_2 .
-
-
-
- P_n with $[A \rightarrow X_1X_2X_3 \dots X_n.y]$ and $uX_1X_2 \dots X_n$.

But this construction also shows that M accepts $uX_1X_2 \dots X_m = w$. QED

Proof: (M recognizes only viable prefixes in G.) Let w be a string accepted by M . By induction on $|w|$, we have:

For $|w| = 0$, M accepts w in the start state, and the empty string is a viable prefix in every grammar.

For $|w| = k + 1$, consider the next-to-last move of M , from some state R to P on some symbol X :

$$w = rX$$

By induction, r is a viable prefix valid for R . Also because of the transition (R to P on X), and lemma 5.2, every item valid for R is in R . There must therefore be an item of the form $[A \rightarrow x.Xy]$ in R such that a derivation

$$S \Rightarrow^* uAz \Rightarrow uxYyz$$

exists, with $ux = r$. But this derivation also shows that $uxX = w$ is a viable prefix and therefore valid for state P . QED

Finiteness of Machine States

We have not required that the number of states in M be finite in these proofs, only that they be countable (induction requires the countable property). The proofs establish a correspondence between the states, the set of items constructed for them, and the (usually infinite) set of viable prefixes in the grammar. However, the state set must be finite, since there are a finite number of possible ways of constructing item sets. We have therefore also indirectly shown that the set of viable prefixes in some grammar are recognized by a finite state automaton.

6.1.4. Resolution of Inadequate States

We now return to the problem of the inadequate states that can arise in the construction of an LR parsing machine. We can deal with these in one of several different ways, as follows:

1. Discard the LR(0) construction and attempt the construction of an LR(k) machine, first for $k = 1$, then $k = 2$, etc. We shall define the construction of an LR(k) machine later. We note here only that the size of each state set and the number of states increases exponentially with k , so that the construction is practical only for small k or a small grammar.

2. A full LR(k) construction can be compromised in the interests of reducing the number of states and state-set size by merging certain states. The result, if successful, is identification of the lookahead set for every inadequate state, as well as generation of the LR machine. This technique yields a so-called *LALR(k)* parser. We shall discuss LR(k) construction in section 6.2.

3. The LR(0) machine itself can be used to infer lookahead sets for the inadequate states. A common algorithm used for the purpose is given next. Essentially, we may infer k -token lookaheads by backtracking the LR(0) machine from an inadequate state. We call such a resolution an *SLALR(k)* machine.

4. The lookaheads may be determined by computing the FOLLOW sets for the nonterminal left member of the production in the inadequate state. When this is done, we have an *SLR(k)* resolution.

These resolution approaches are ranked in order of decreasing strength. Thus there are grammars that yield to an SLALR treatment, but not an LALR, etc. Nevertheless, even the SLR(k) resolution approach is sufficiently powerful to be useful for practical grammars and is the easiest to implement.

SLR stands for “simple LR”, and was first reported by DeRemer [1971]. *LALR* stands for “lookahead LR”, and is described in Aho and Johnson [1974].

We defer the two LR(k) resolutions (for $k > 0$) until a later section, as it requires considerable development. We must first introduce a more general definition of a state-set item and a more general construction system.

SLALR(1) Resolution

The inadequate states may be resolved by computing a set of lookaheads associated with transitions in the LR(0) machine, using the machine transitions themselves. Although weaker than a merged-state LR(k) parser, it is more powerful than an SLR(k) resolution. As usual, we consider only $k = 1$.

1. Let P be some inadequate state in an LR(0) machine. It therefore contains at least one completed item in addition to some others of any kind.

2. For each completed item in P , create a new state P' and a set of lookahead transitions (as yet unlabeled with symbols) from P to P' . Associate the completed item with P' , removing it from P . A lookahead transition is similar to a read transition, except that the read head is not moved.

Figure 6.5 illustrates steps 1 and 2. State 12 has two read transitions, on T and on F , to states 15 and 17, respectively. It is also associated with the completed items $[F \rightarrow \cdot]$ and $[E \rightarrow ET \cdot]$. Then steps 1 and 2 result in figure 6.5(b), in which new states 12' and 12'' are created to accept the completed items. The labels $L1$ and $L2$ represent a set of lookahead symbols (yet to be determined) for the new states.

We now have only apply states, each containing a single completed item, and read states, each possessing only read or lookahead transitions to other states. Although this machine is not exactly the same as that shown in figure 6.1, it may readily be transformed into it.

We next associate a *lookahead set*, consisting of terminal symbols, with each transition as follows.

3. The lookahead set for a terminal read transition is the symbol associated with the transition.

4. Consider a nonterminal read transition T to some state P . Its lookahead set is the union of the lookahead sets into P (if P is an apply state) or the transitions out of P (if P is a read state).

5. Let R be an apply state with production $A \rightarrow w$, and let Q be a state with a lookahead transition $Q' \rightarrow R$. Let $P(R)$ be the set of read states in M such

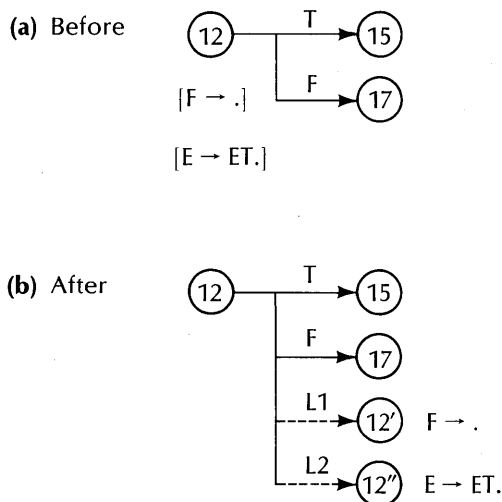


Figure 6.5. Splitting an inadequate state.

that for each X in $P(R)$, configuration $(X, w) \vdash^* (Q, \epsilon) \vdash (R, \epsilon)$. That is, we find all the states X such that the machine can accept w by starting from X and ending in R , by passing through Q .

Now there must be a transition T from X on the nonterminal symbol A (A is part of the production associated with R). This transition is guaranteed by the construction process. Add the lookahead set $L(T)$ associated with T to the set $L(Q')$, for every X .

6. Steps 4 and 5 are repeated until no additions to any lookahead set can be made anywhere in the machine. At that point, the lookahead transitions into the apply state carry the desired SLALR(1) lookahead sets.

Example Construction. The following grammar generates an LR(0) machine with several inadequate states:

1. $S \rightarrow E \perp$
2. $E \rightarrow E;T$
3. $E \rightarrow T$
4. $T \rightarrow Ta$
5. $T \rightarrow \epsilon$

The LR machine for this grammar is shown in figure 6.6. It contains four inadequate states, 6, 8, 9, and 10. Lookahead transitions to apply states have been added in the figure, as demanded by steps 1 and 2 in the SLALR(1) algorithm.

Steps 4 and 5 are repeatedly applied to yield the decorated machine of figure 6.7. A suitable starting point is the E transition (6,7). By step 4, this transition has the lookahead set $\{\perp ;\}$, from the transitions from state 7. Similarly, the T transitions (6,8) and (9,10) both include the set $\{a\}$. Eventually, these transitions will also pick up the lookahead sets from the transitions (10,2) and (8,3), but they are empty so far.

Step 5 applied to state 2 causes this state to pick up the symbols associated with the E transition (6,7), since only state 6 (through 7, 9, 10, 2) leads to state 2 on the string “ $E;T$ ”. These symbols $\{\perp ;\}$ are then associated with the transition (10,2), and also with transition (9,10).

States 5 and 5' are both associated with the production $T \rightarrow \epsilon$, and eventually become a single state in the final machine. However, the transitions into them are distinct. It can happen that the trace-back caused by step 5 can yield different lookahead sets for the two states, and therefore different lookahead sets for the transitions. The same state may therefore be accessed by different lookahead transitions.

We find that this grammar is SLALR(1). The test is whether the lookahead sets and the terminal read sets from each state are pair-wise disjoint, and they are for this machine. The lookahead sets we developed for the nonterminal transitions are not considered in the test.

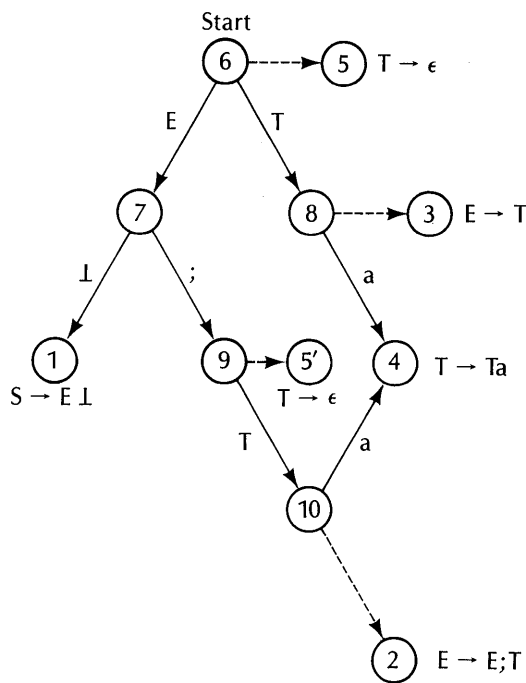


Figure 6.6. LR machine for an example grammar. The grammar productions are attached to states in the machine.

SLR(1) Resolution

Recall that a state P is inadequate if it contains a completed item $[A \rightarrow w.]$ and some other item, of any kind. The presence of $[A \rightarrow w.]$ in a state P means that there is some set of derivations of the form

$$S \Rightarrow^* xAy \Rightarrow xwy$$

such that the LR(0) machine accepts xw by falling into state P . Clearly, the set $L = \text{FIRST}_1(y \text{ FOLLOW}_1(S))$ is the desired lookahead set for this item in P . However, L is a subset of $\text{FOLLOW}_1(A)$, clearly. The latter set is easy to determine, and if the use of FOLLOW is sufficient to resolve every inadequate state (it often is), then we have an SLR(1) resolution. A resolution of inadequate states in an LR(0) machine using FOLLOW_k would be an SLR(k) resolution.

For example, consider the machine of figure 6.1, with the items developed as above. State 10 is inadequate, since it contains the two items

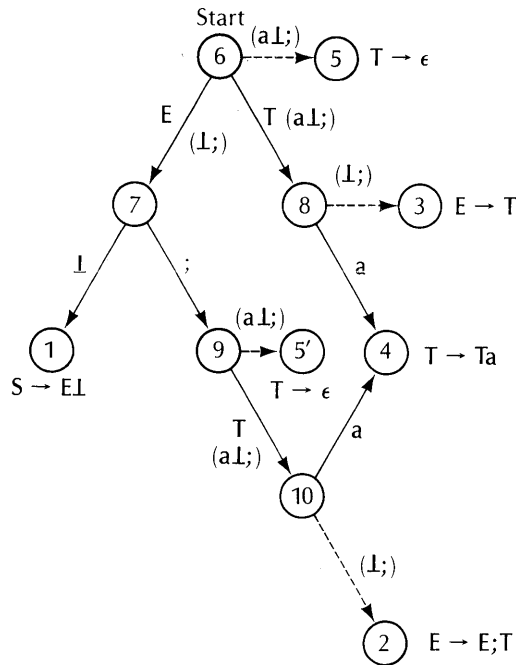


Figure 6.7. Machine of figure 6.6 decorated with lookahead sets.

$[E \rightarrow T.]$
 $[T \rightarrow T.*F]$

The set $\text{FOLLOW}(E)$ is $\{), +, \perp\}$, as is easily shown from the grammar. (See chapter 4 for a complete discussion.) This set does not contain symbol “*”, hence state 10 may be resolved by a one symbol lookahead. It is shown in figure 6.1 as a new state, 16, with two lookahead transitions to states 10 and 3.

Similarly, state 15, which contains the items

$[E \rightarrow E + T.]$
 $[T \rightarrow T.*F]$

may also be resolved with the same $\text{FOLLOW}(E)$ set. It is shown in figure 6.1 as a new state, 17, with lookahead transitions to states 2 and 15.

The use of the FOLLOW function may or may not resolve an inadequate state. If it fails, the grammar may still be SLALR(1), LALR(1) or LR(1). Essentially, the FOLLOW function yields all the symbols that can follow some nonterminal A in a sentential form. It makes no distinctions based on the particular inadequate state that we are interested in. The FOLLOW

function therefore tends to yield a larger set of lookahead symbols in general than can actually exist for that nonterminal in that state.

Exercises

1. Consider the grammar G_3

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow Aa \mid dAb \mid dca \mid cb \\ A &\rightarrow c \end{aligned}$$

of Aho and Ullman (Aho [1972]). Show that it is SLALR(1) but not SLR(1).

2. Devise an efficient data structure in Pascal for a set of productions and the item-sets. For example, an item may be characterized by the number pair (i, j) , where i is a production number and j is the mark position within the pair. What else might be added for the sake of an efficient LR(0) state set determination? Sketch the operations required for a general purpose parser generator.

6.2. LR(k) Parsers

We now describe a more general means of constructing an LR(k) parser and also introduce the concept of canonical tables and a canonical parser. These are useful in obtaining several important reductions in parser tables and are also referred to in many papers on LR(k) systems.

An $LR(k)$ item is a marked production and a terminal string w in Σ^* . The length $|w|$ is at most k , and w is called a *lookahead string*. We denote such an item by:

$$[A \rightarrow x.y, w]$$

The marked production $A \rightarrow x.y$ is called the *core* of the item. Several items with the same core may be combined through the notation

$$[A \rightarrow x.y, \{u_1, u_2, \dots, u_n\}]$$

which stands for the set of items

$$\begin{array}{l}
[A \rightarrow x.y, u_1] \\
[A \rightarrow x.y, u_2] \\
\vdots \\
[A \rightarrow x.y, u_n]
\end{array}$$

We are interested in a subset of all possible items. In particular, we will be constructing sets of items in such a way that for each item of the form $[A \rightarrow x.y, u]$, there exists some right-most derivation of the form

$$S \Rightarrow^* rAst \Rightarrow^* rxyst \Rightarrow^* r'x'ut$$

where

$$rx \Rightarrow^* r'x' \quad \text{and}$$

$$ys \Rightarrow^* u$$

As a rationale for this form of item, consider a bottom-up LR(k) parser with rx in the stack and ut next in the input list. String u is clearly a k -symbol lookahead, and among the productions that are potential candidates for the parse so far, $A \rightarrow xy$ is certainly one. If this production is next to be reduced, the lookahead string u , along with the current state, will indicate production $A \rightarrow xy$.

We see that by carrying along a lookahead string in the items when the parser state set is constructed, we will automatically have the necessary lookahead strings available. A k -symbol lookahead will be used on both read and reduce actions, for the sake of generality. This lookahead will always appear in an item-set as the lookahead string. It can be shown that an LR(k) constructor accepts the broadest class of grammars, larger than the SLR(k) or SLALR(k) constructors described earlier.

The LR(k) item-set construction proceeds in three steps as follows:

1. *Initial state construction:*

(a) If $S \rightarrow w$ is a production, then add $[S \rightarrow .w, \epsilon]$ to the initial state. Symbol S is the grammar's start symbol.

(b) If $[A \rightarrow .Bx, w]$ is in the initial state, and $B \rightarrow y$ is a production, then add all the items of the form $[B \rightarrow .y, z]$ to the initial state, where z is in the set $\text{FIRST}_k(xw)$. Of course, this is added to the set only if it is not already there. We repeat this step until no more items can be added to the initial state.

2. *Starting a new state:* Let $[A \rightarrow r.Xy, w]$ be an item in some state P , where X is a terminal or nonterminal token. We locate every item in state P such that X follows the mark “.”, then start a new state Q with these items, but with the mark moved past X . Thus item $[A \rightarrow r.Xy, w]$ will become item $[A \rightarrow rX.y, w]$ in state Q . The set of items so used to start state Q is called the *core set* of Q .

The core set of Q may be identical to the core set of some other state Q' already constructed. If so, states Q and Q' are equivalent and may be merged. By merging such states, we guarantee that the construction process will terminate in a finite number of steps.

A transition from P to Q on X will be part of the $LR(k)$ parser, thanks to this operation.

3. *Closing a new state:* For every item of the form $[A \rightarrow x.By, w]$ in a state Q , and for every production $B \rightarrow z$, we add to Q all the items of the form $[B \rightarrow .z, u]$, where u is in the set $FIRST_k(yw)$. This step is repeated until no more items can be added to Q . After step 1 is finished, steps 2 and 3 are repeated alternately, in that order, until no more new states can be constructed or closed.

As before, some states will carry completed items, of the form $[A \rightarrow x., w]$. For them, a reduction is indicated in that state, with the production $A \rightarrow x$. Now there will in general be several items in that state of the form $[A \rightarrow x., \dots]$. We therefore have a set of lookahead strings W such that the reduction is permitted only if the next k (or less) tokens in the input list is among the set W . The lookaheads may also be used to resolve one of several possible reductions or to resolve a conflict between a reduction and one or more read operations.

It can be shown that the grammar is $LR(k)$ if and only if the following condition is met by the state set constructed as above:

For every state P containing an item of the form $[A \rightarrow x., w]$, there exists no other item of the form $[B \rightarrow y.z, w]$ in P .

6.2.1. Canonical $LR(1)$ Parsing Tables

For the sake of ease of comprehension, we specialize k to 1 in most of the following arguments. The extension to $k > 1$ is relatively straightforward, but tends to clutter up the notation.

A *canonical $LR(1)$ parser* is a pair of functions f and g representing a finite control for a push-down automaton, as follows:

Function f. The *action function f* maps a pair $\{T, Y\}$, where T is a table name and Y is a member of $\Sigma \cup \{\epsilon\}$ into one of the following:

- APPLY n , where n is a production number.
- SHIFT, abbreviated “S”.
- ACCEPT, i.e., “halt”, abbreviated “A”.
- ERROR, abbreviated “X”.

The APPLY action will appear in the table as the production number. Thus an action table entry will be a production number, S, A, or X. We shall later introduce another entry, “@”, that will mean *inaccessible*.

Each table T is associated with one state P generated in the LR(1) construction process. Then for every completed item of the form $[A \rightarrow x., w]$ in state P , we set $f(T, w) = n$, where n is the number of the production $A \rightarrow x.$ For every item of the form $[B \rightarrow u.xv, w]$ in state P , where $|x| = 1$, and x is a terminal token, we set $f(T, w) = S$, i.e., *shift*. If item $[G \rightarrow x., w]$ appears in state P , where G is the grammar’s goal symbol, we set $f(T, w) = A$, i.e., *accept*. All other f entries for table T are set to X, i.e., *error* (although not all these can necessarily be reached with some input string).

Function g. The *goto function g* maps a pair $\{T, Y\}$, where T is a table name and Y is a member of $N \cup \Sigma \cup \{\epsilon\}$ into

- A table name T' or
- ERROR, abbreviated “X”.

Now table T is associated with some state P in the construction process. For every item of the form $[A \rightarrow u.xv, w]$ in that state, where $|x| = 1$, we set $g(T, w) = T'$, where table T' is associated with a state Q containing the item $[A \rightarrow ux.v, w]$, and such that a transition from P to Q on symbol x exists. Note that such items cause *shift* to be entered in the appropriate column of the f function, if x is terminal. Here, however, x may be terminal or nonterminal. All other g entries in table T are ERROR.

6.2.2. A Canonical Parser

An LR(1) parser based on the functions f and g consists of a finite control (represented by the following Pascal program) and a stack containing table names. Let T_0 be an initial table that corresponds to the initial state constructed. The parser may then be described by the following program. The functions in the program are explained through comments.

```
var X: char;    {an input token}
```

```

START: PUSH(T0); {push the initial table name in the stack}
      X:=NEXTTOKEN;
          {variable X gets the next token}

while ~ f(TOS, X) = "A" do
    {TOS is the table on the top of the stack. We continue
    this algorithm until the accept condition is found}
begin {accept state not reached if we enter this block}

    if f(TOS, X) = "S" then
        begin {a shift operation is wanted}

            PUSH(g(TOS,X)); {push the table indicated by the
                             goto table, based on token X}
            X:=NEXTTOKEN; {get the next token}
        end
    else
        if f(TOS, X) = reduce n then {n is a production number}
            begin {a reduce operation is wanted}

                APPLY(n); {indicate to the rest of the compiler that
                           production n is next to be applied}
                POP(RIGHTMEMLEN(n));
                    {pop as many elements from the
                     stack as tokens in the right member of
                     production n}
                PUSH(g(TOS, LEFTMEM(n)))
                    {some table name was exposed
                     through the previous POP operation; this is
                     used in the g function, along with the
                     nonterminal left member associated with
                     production n, to push another table name}
            end
        else ERROR {syntax error}
    end
end

```

The simplicity of this parser speaks for itself. We see from the reduce operation that the *g* function must contain mappings associated with nonterminal tokens in general. The shift operation implies that a *g*-mapping associated with terminal tokens is needed. When the end of the input list is reached, there may be several reduce moves left or some error indication; we therefore see that the action function *f* must include the empty string ϵ in its domain. Note that this parser does not require a special termination token.

Example. Let us construct an LR(1) canonical parser for grammar G_0 , as follows:

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow a$

We need the $FIRST_1$ sets for the nonterminals, as follows:

nonterminal v	$FIRST_1(v)$
E	(a
T	(a
F	(a

The initial state 0 contains the items $[E \rightarrow .E + T, \epsilon]$ and $[E \rightarrow .T, \epsilon]$, through construction rule 1(a). The completion rule on item $[E \rightarrow .E + T, \epsilon]$ implies that each of the following items must be included in state 0:

- $[E \rightarrow .E + T, +]$, from $FIRST_1(+T)$
- $[E \rightarrow .T, +]$, from production $E \rightarrow T$
- $[T \rightarrow .T * F, +]$, from production $T \rightarrow T * F$
- $[T \rightarrow .F, +]$, from production $T \rightarrow F$
- $[T \rightarrow .T * F, *]$, from item $[T \rightarrow .T * F, +]$ and $FIRST_1(*F +)$

Sixteen items eventually appear in state 0. This state and the remaining states in the complete LR(1) state set are given below. The core items are marked ">".

State	Item	Lookaheads	Go to state
0	> $E \rightarrow .E + T$	$\epsilon +$	1
	> $E \rightarrow .T$	$\epsilon +$	2
	$T \rightarrow .T * F$	$\epsilon + *$	2
	$T \rightarrow .F$	$\epsilon + *$	3
	$F \rightarrow .(E)$	$\epsilon + *$	5
	$F \rightarrow .a$	$\epsilon + *$	4
1	> $E \rightarrow E. + T$	$\epsilon +$	6
2	> $E \rightarrow T.$	$\epsilon +$	reduce (2)
	> $T \rightarrow T.*F$	$\epsilon + *$	7

3	> $T \rightarrow F.$	$\epsilon + *$	reduce (4)
4	> $F \rightarrow a.$	$\epsilon + *$	reduce (6)
5	> $F \rightarrow (.E)$	$\epsilon + *$	8
	$E \rightarrow .E + T$	) +	8
	$E \rightarrow .T$	) +	9
	$T \rightarrow .T * F$	) + *	9
	$T \rightarrow .F$	) + *	10
	$F \rightarrow .(E)$	) + *	12
	$F \rightarrow .a$	) + *	11
6	> $E \rightarrow E + .T$	$\epsilon +$	13
	$T \rightarrow .T * F$	$\epsilon + *$	13
	$T \rightarrow .F$	$\epsilon + *$	3
	$F \rightarrow .(E)$	$\epsilon + *$	5
	$F \rightarrow .a$	$\epsilon + *$	4
7	> $T \rightarrow T * .F$	$\epsilon + *$	14
	$F \rightarrow .(E)$	$\epsilon + *$	5
	$F \rightarrow .a$	$\epsilon + *$	4
8	> $F \rightarrow (E.)$	$\epsilon + *$	15
	> $E \rightarrow E + T$	) +	16
9	> $E \rightarrow T.$	) +	reduce (2)
	> $T \rightarrow T * F$	) + *	17
10	> $T \rightarrow F.$	) + *	reduce (4)
11	> $F \rightarrow a.$	) + *	reduce (6)
12	> $F \rightarrow (.E)$	) + *	18
	$E \rightarrow .E + T$	) +	18
	$E \rightarrow .T$	) +	9
	$T \rightarrow .T * F$	) + *	9
	$T \rightarrow .F$	) + *	10
	$F \rightarrow .(E)$	) + *	12
	$F \rightarrow .a$	) + *	11
13	> $E \rightarrow E + T.$	$\epsilon +$	reduce (1)
	> $T \rightarrow T * F$	$\epsilon + *$	7
14	> $T \rightarrow T * F.$	$\epsilon + *$	reduce (3)

15	> $F \rightarrow (E).$	$\epsilon + *$	reduce (5)
16	> $E \rightarrow E + .T$	) +	19
	$T \rightarrow .T * F$	) + *	19
	$T \rightarrow .F$	) + *	10
	$F \rightarrow .(E)$	) + *	12
	$F \rightarrow .a$	) + *	11
17	> $T \rightarrow T * .F$	) + *	20
	$F \rightarrow .(E)$	) + *	12
	$F \rightarrow .a$	) + *	11
18	> $F \rightarrow (E.)$	) + *	21
	> $E \rightarrow E. + T$	) +	16
19	> $E \rightarrow E + T.$	) +	reduce (1)
	> $T \rightarrow T * F.$	) + *	17
20	> $T \rightarrow T * F.$	) + *	reduce (3)
21	> $F \rightarrow (E).$	) + *	reduce (5)

The canonical tables found from these state-sets are as follows:

	action, f						goto, g							
	a	+	*	(	)	ϵ	E	T	F	a	+	*	(	)
0	S	X	X	S	X	X	1	2	3	4	X	X	5	X
1	X	S	X	X	X	A	X	X	X	X	6	X	X	X
2	X	2	S	X	X	2	X	X	X	X	X	7	X	X
3	X	4	4	X	X	4	X	X	X	X	X	X	X	X
4	X	6	6	X	X	6	X	X	X	X	X	X	X	X
5	S	X	X	S	X	X	8	9	10	11	X	X	12	X
6	S	X	X	S	X	X	X	13	3	4	X	X	5	X
7	S	X	X	S	X	X	X	X	14	4	X	X	5	X
8	X	S	X	X	S	X	X	X	X	X	16	X	X	15
9	X	2	S	X	2	X	X	X	X	X	X	17	X	X
10	X	4	4	X	4	X	X	X	X	X	X	X	X	X
11	X	6	6	X	6	X	X	X	X	X	X	X	X	X
12	S	X	X	S	X	X	18	9	10	11	X	X	12	X
13	X	1	S	X	X	1	X	X	X	X	X	7	X	X

	action, f						goto, g							
	a	+	*	(	)	ε	E	T	F	a	+	*	(	)
14	X	3	3	X	X	3	X	X	X	X	X	X	X	X
15	X	5	5	X	X	5	X	X	X	X	X	X	X	X
16	S	X	X	S	X	X	X	19	10	11	X	X	12	X
17	S	X	X	S	X	X	X	X	20	11	X	X	12	X
18	X	S	X	X	S	X	X	X	X	16	X	X	21	
19	X	1	S	X	1	X	X	X	X	X	17	X	X	
20	X	3	3	X	3	X	X	X	X	X	X	X	X	X
21	X	5	5	X	5	X	X	X	X	X	X	X	X	X

The following is a trace of the LR(1) parser for the string “a+a*a”:

stack	X (next token)	input	action
0	a	+a*a	S
0,4	+	a*a	6 F→a
0,3	+	a*a	4 T→F
0,2	+	a*a	2 E→T
0,1	+	a*a	S
0,1,6	a	*a	S
0,1,6,4	*	a	6 F→a
0,1,6,3	*	a	4 T→F
0,1,6,13	*	a	S
0,1,6,13,7	a	ε	S
0,1,6,13,7,4	ε		6 F→a
0,1,6,13,7,14	ε		3 T→T*F
0,1,6,13	ε		1 E→E+T
0,1	ε		A (accept)

6.2.3. Table Reductions

We shall now explore several kinds of reductions that may be made on a set of canonical LR(1) tables. (These reductions also apply to canonical LR(k) tables for $k > 1$. An LR(k) table maps a string of length k to reduce, shift, etc.)

The following discussion will be based on the canonical parser given earlier in this section.

The general strategy is to first identify those entries that are inaccessible. They will be some of the “X” entries, which will be marked “@”. Since these entries can never be reached on scanning any input string (whether in the language or not), they can in fact be made anything at all. We can then redefine them in an attempt to reduce the number of tables.

One reduction can be made through *error postponement*. Here we look for X entries in the f table such that, when replaced by some *reduce*, permits the merger of two or more states without affecting the detection of the error ultimately. Essentially, we allow some more spurious reduce operations, rather than report the syntax error immediately, but require that at some future point in the parse the error associated with the next token will be detected and reported. We will not allow some sequence of reduce operations to end in a state that can accept the next error token.

Another reduction is possible through the removal of transitions connected to single productions. A single production has the form $A \rightarrow B$, where B is a nonterminal. Usually, single productions require no action on the part of the rest of the compiler; for example, in building the abstract syntax tree, those nodes associated with a production $A \rightarrow B$ need not be added. The result of removing state transitions associated with single productions is often a sizable reduction in the canonical table, and a concomitant improvement in parser speed.

Each of these reductions will in general yield some new canonical table set containing inaccessible states or pairs of equivalent states. The inaccessible states may be discarded and the equivalent states merged. When the reduced canonical table is obtained, it may easily be transformed into a set of packed sparse matrix tables, an example of which is given in figure 6.3.

Identification of Inaccessible Entries

Here, we wish to identify every X entry in a canonical table that can never be reached.

We first demand that a syntax error be detected as early as possible. The canonical parser algorithm clearly does this by inspecting $f(T, h)$, where h is the next input token and T is the current table (or state). If $f(T, h)$ is neither accept, shift nor reduce, then it reports an error. We say that token h is an *error token*; the string w preceding h is such that there exists a set of strings wy in the language accepted by the parser, but h is not $FIRST_1(y)$ for any y.

Since the parser will always first detect an error in the f table, it follows that none of the X entries in the g table are accessible, and each of these may therefore be changed to "@".

Now let us examine the X entries in the f table. Let T be a table and h some token, such that $f(T, h) = X$. The parser clearly will be able to access $f(T, h)$ only if one of these conditions holds: (1) T is the initial table T_0 , or (2) there is some table T' and token h' such that $f(T', h') = \text{shift}$ and $g(T', h') = T$.

Case (1) applies upon first entering the parser. Table T is the initial table, and the algorithm must be prepared to report error on any possible first token.

Case (2) says that $f(T, h)$ can be accessed if table T can be reached after